Neural Networks For Sequences

Nebius Academy

Plan

- Recurrent Neural Networks (RNNs)
    - Recurrent Layer
    - Training of RNNs
    - Embedding layer and tokenization
    - Language Modelling with RNNs
- Practice!

Recurrent Neural Networks (RNNs)

Sequence processing

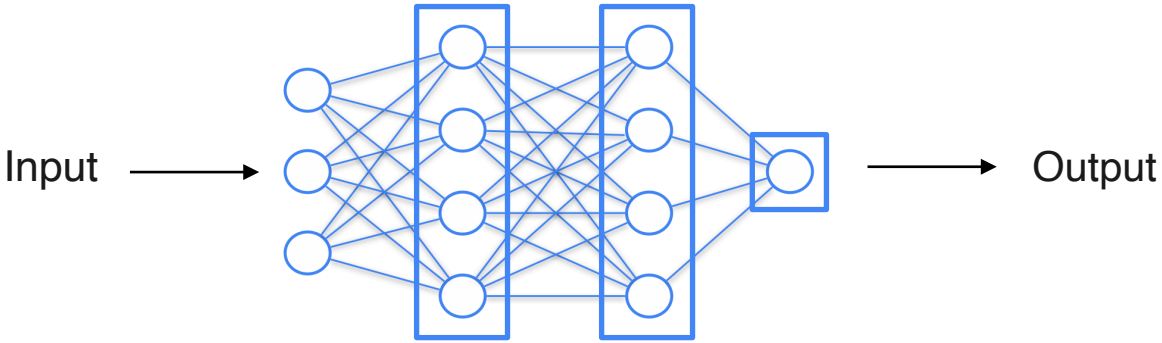
The difference between text/sound and other types of data (for example, images) is the presence of a *time component*.

We read text not instantly, but word by word, in a strictly defined order.

It would be nice to come up with an idea of a neural network that would take into account this feature of these types of data.

Recurrent Layer

How ***fully-connected*** neural network works:

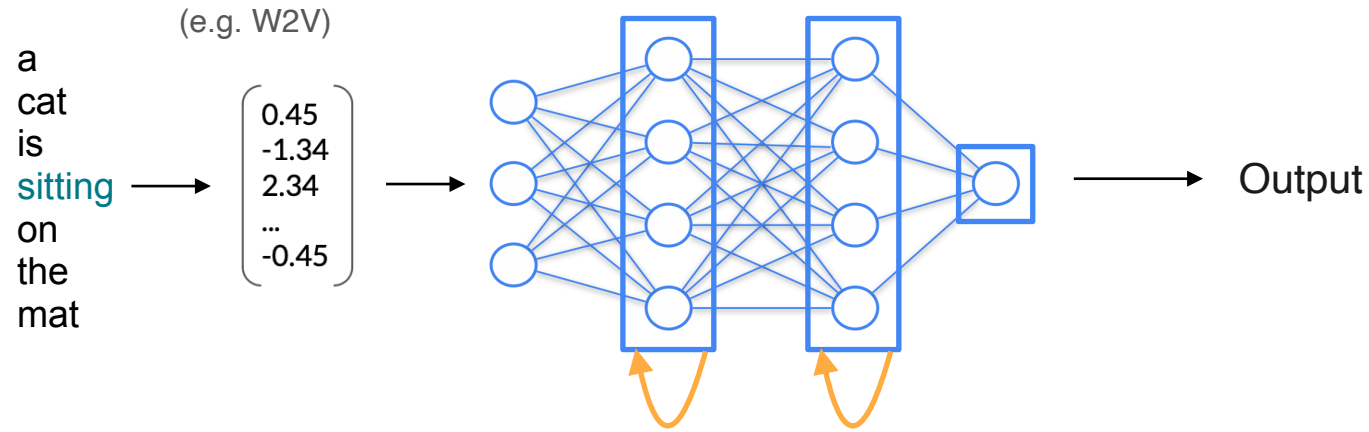


Recurrent Layer

How ***recurrent*** neural network works:

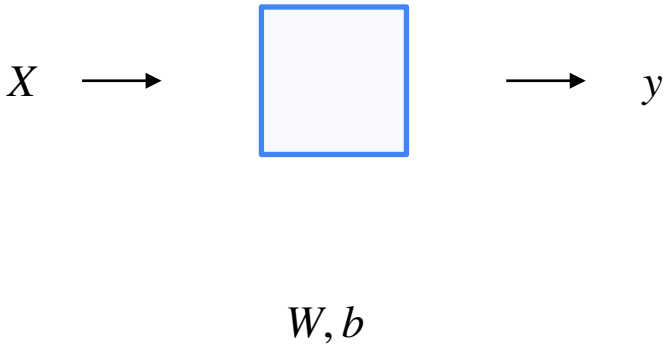
A recurrent neural network processes one token of text at a time.

A “from itself to itself” connection is added to the layer — it is the “memory” of the layer



Recurrent Layer

Fully-connected layer:

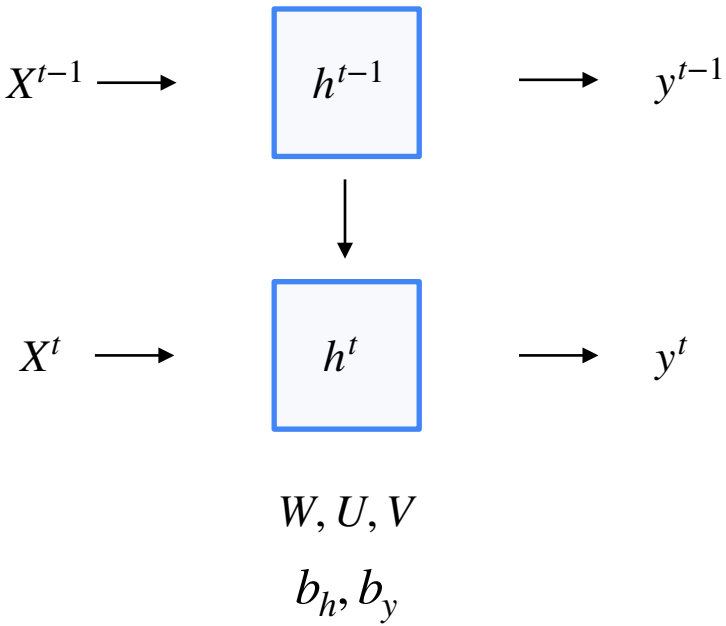


Layer output:

$$y = \sigma(WX + b)$$

Recurrent Layer

Recurrent layer:



Update hidden state:

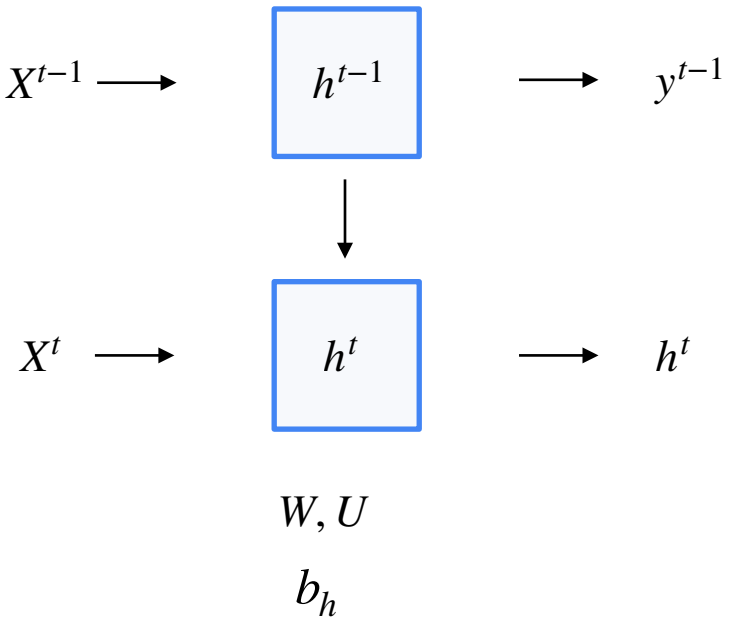
$$h^t = \sigma(WX^t + Uh^{t-1} + b_h)$$

Layer output:

$$y^t = \sigma(Vh^t + b_y)$$

Recurrent Layer

Recurrent layer:



Update hidden state:

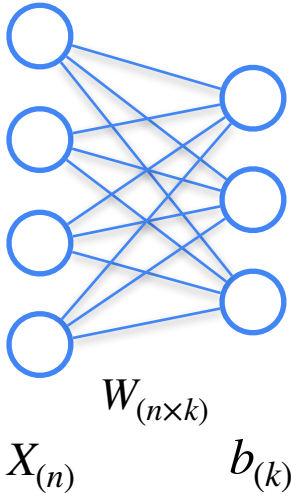
$$h^t = \sigma(WX^t + Uh^{t-1} + b_h)$$

Layer output:

$$y^t = h^t$$

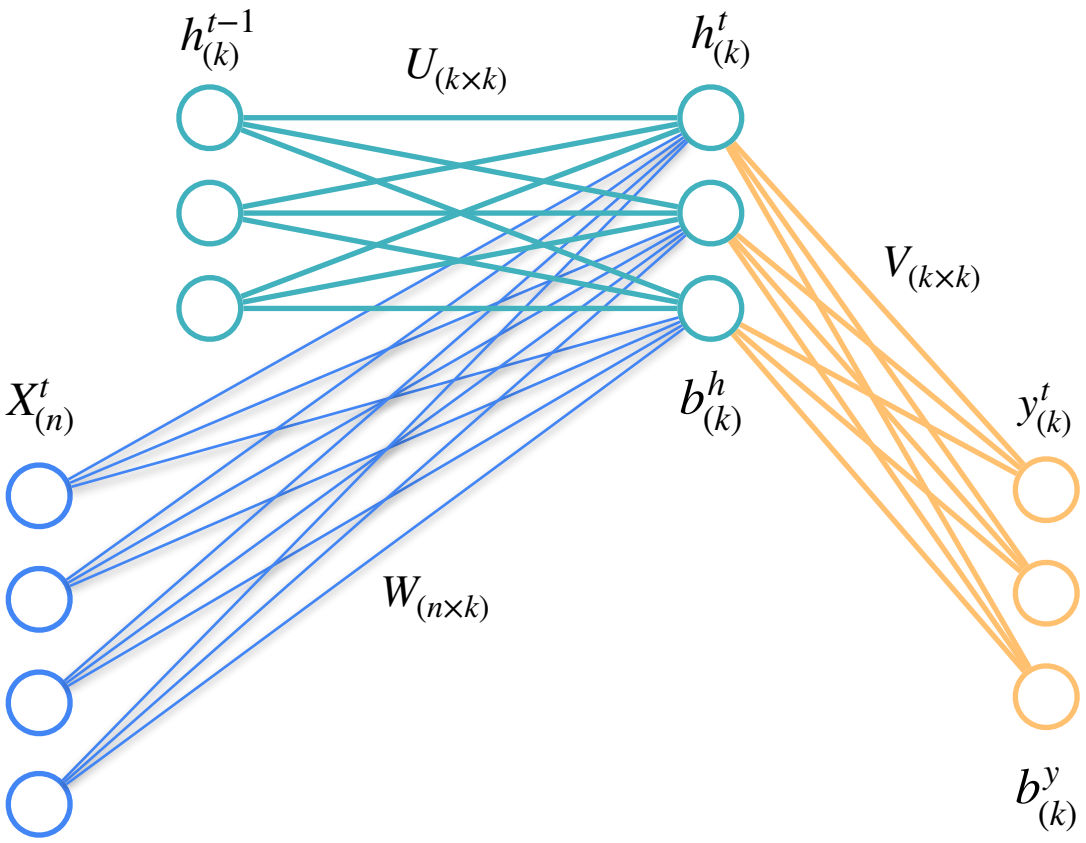
Recurrent Layer

Fully-connected layer:



Layer output:

$$y = \sigma(WX + b)$$



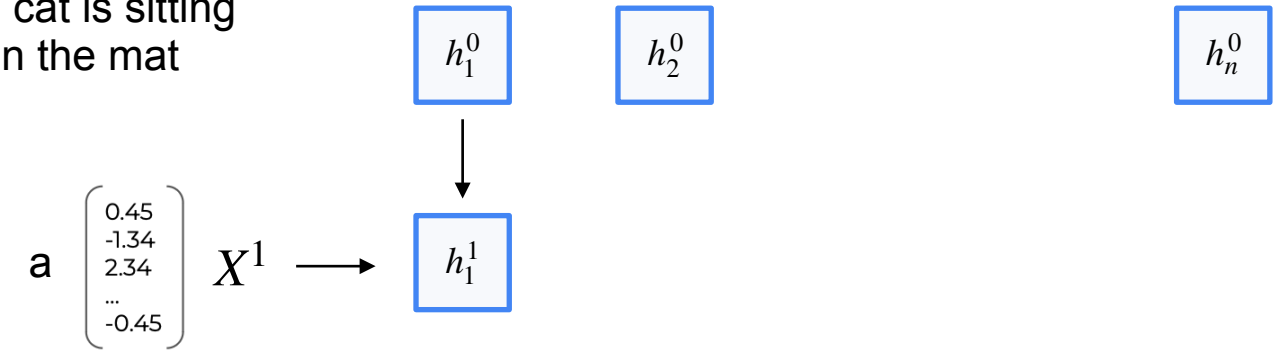
Update hidden state:

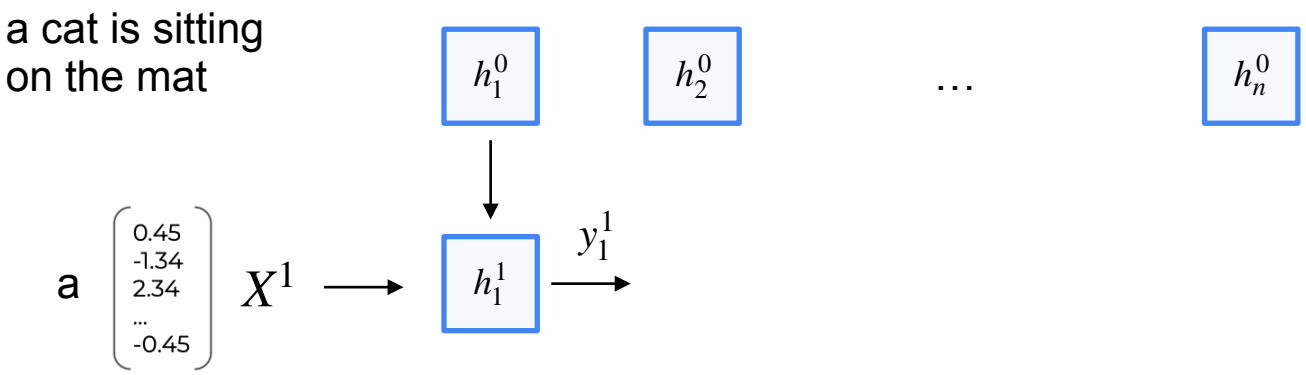
$$h^t = \sigma(WX^t + Uh^{t-1} + b_h)$$

Layer output:

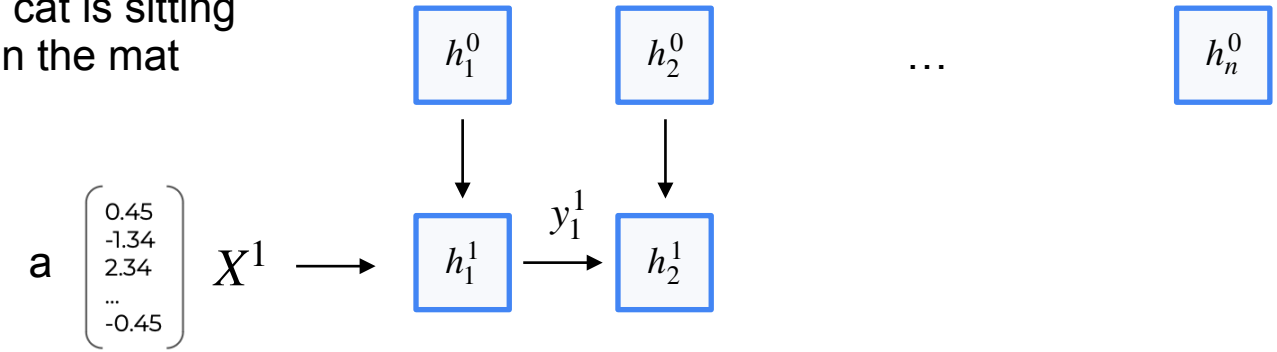
$$y^t = \sigma(Vh^t + b_y)$$

a cat is sitting
on the mat

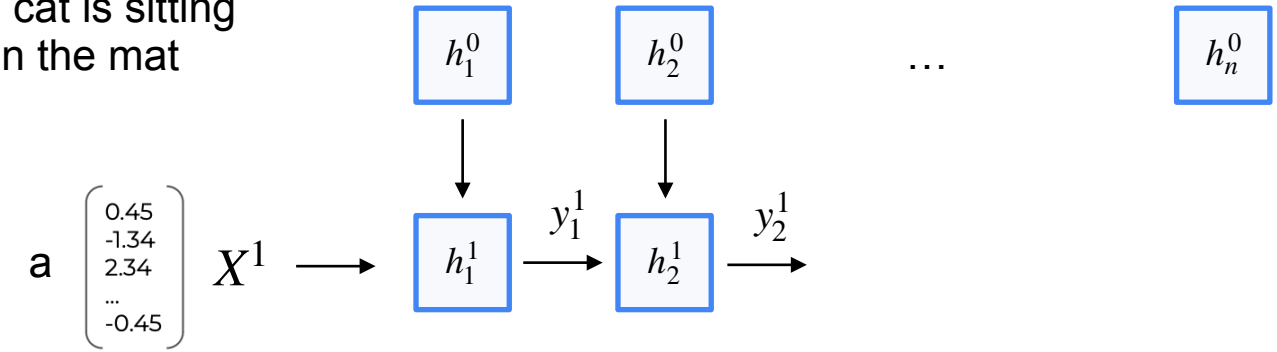



a cat is sitting
on the mat
h_1^0
h_2^0
…
h_n^0
a
0.45
-1.34
2.34
...
-0.45
X^1
h_1^1
y_1^1

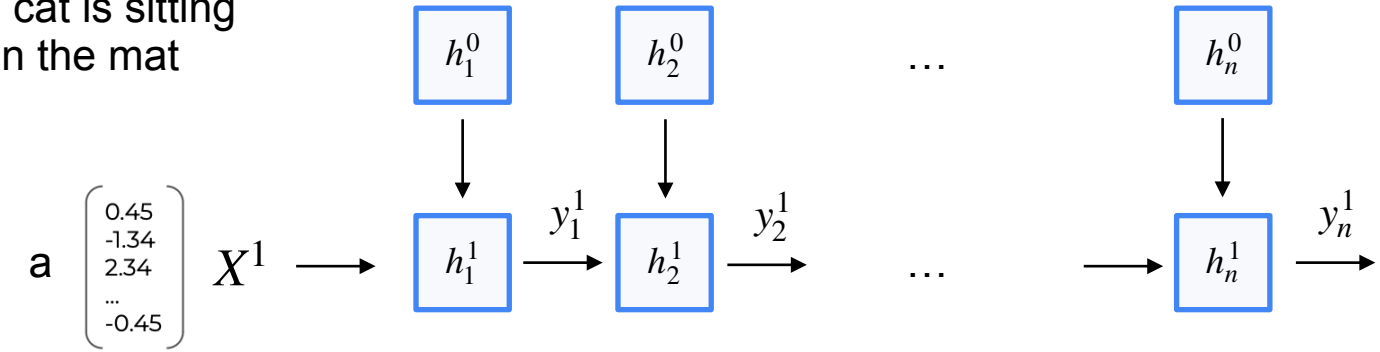
a cat is sitting
on the mat



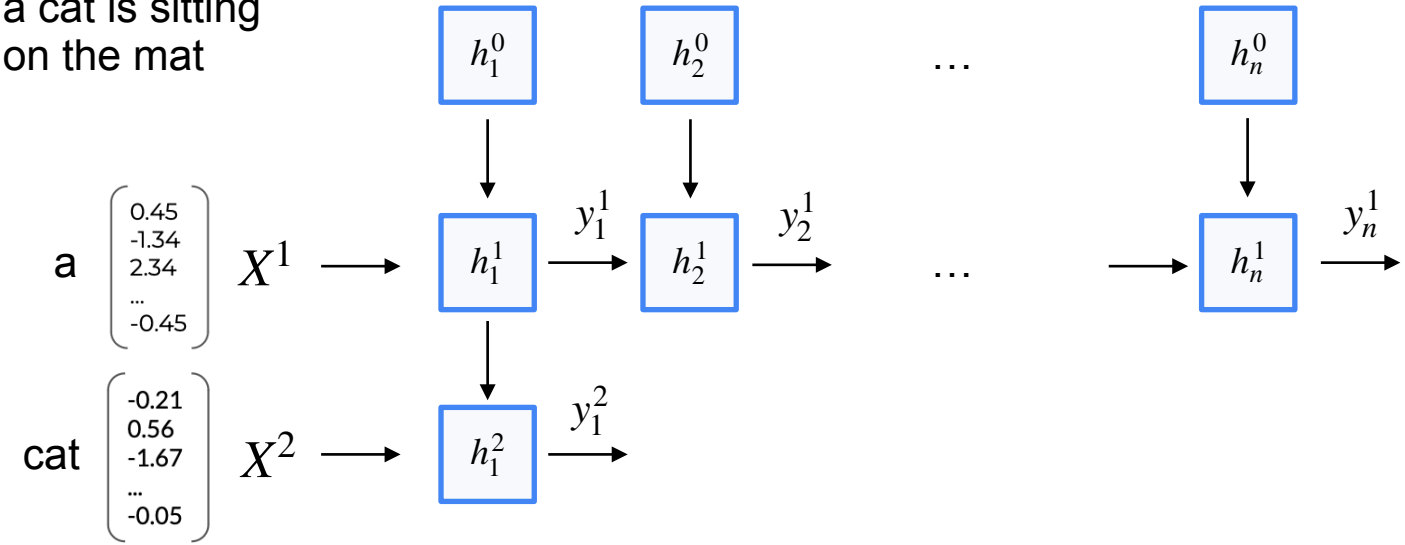
a cat is sitting
on the mat



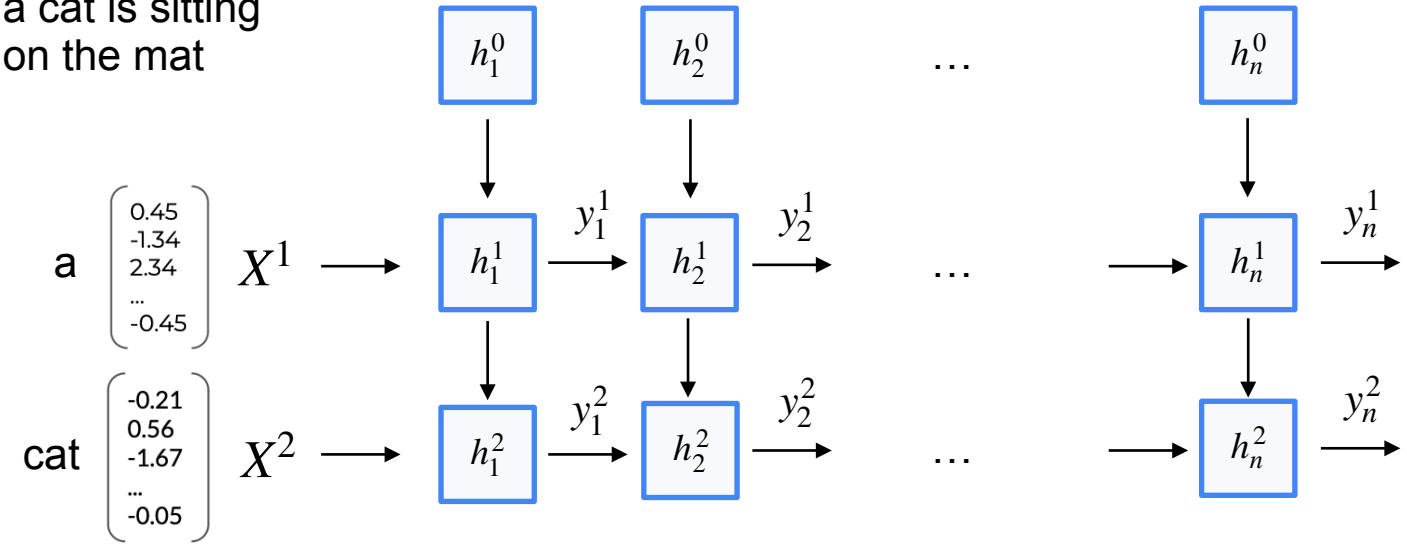
a cat is sitting
on the mat

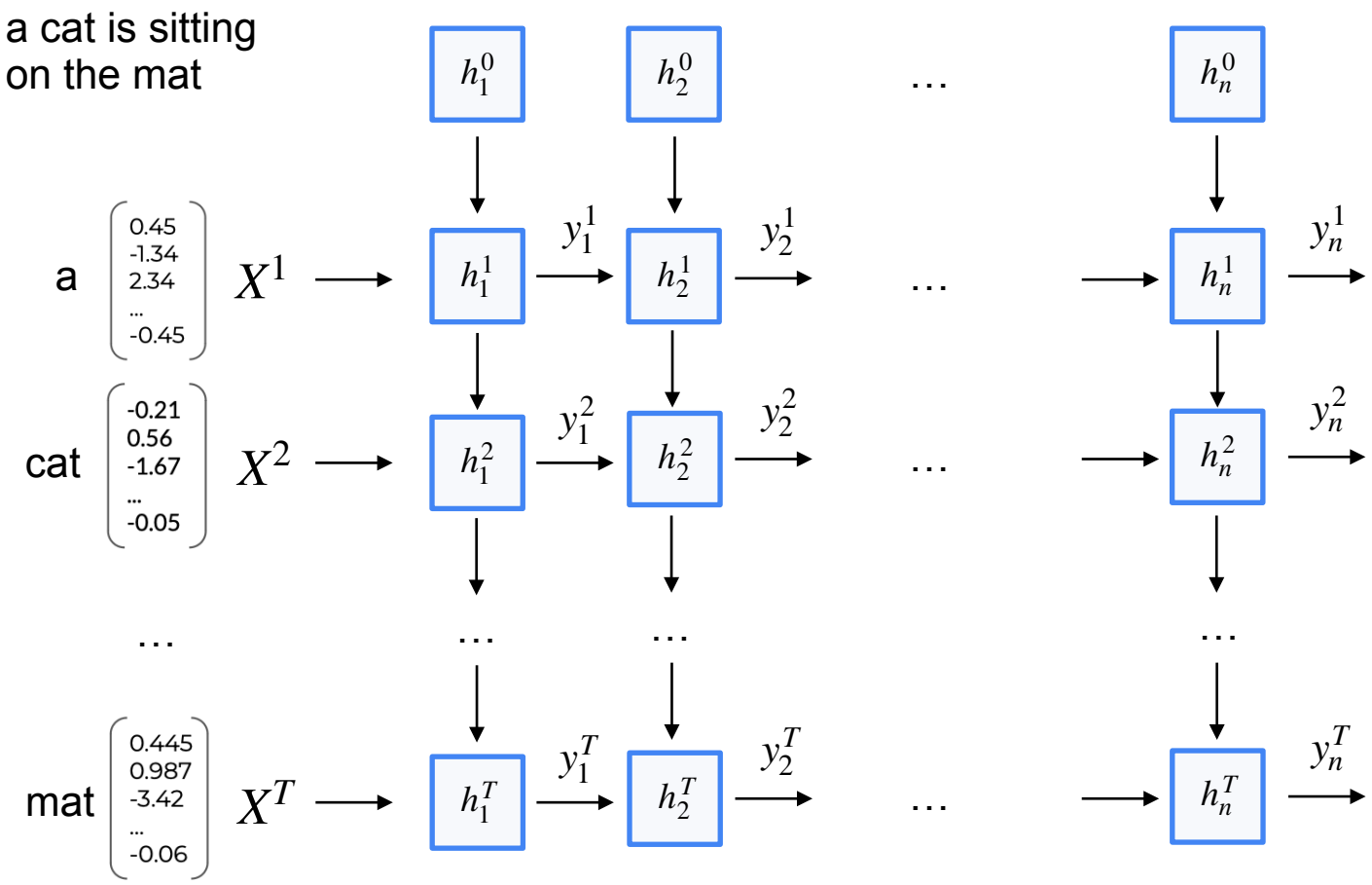


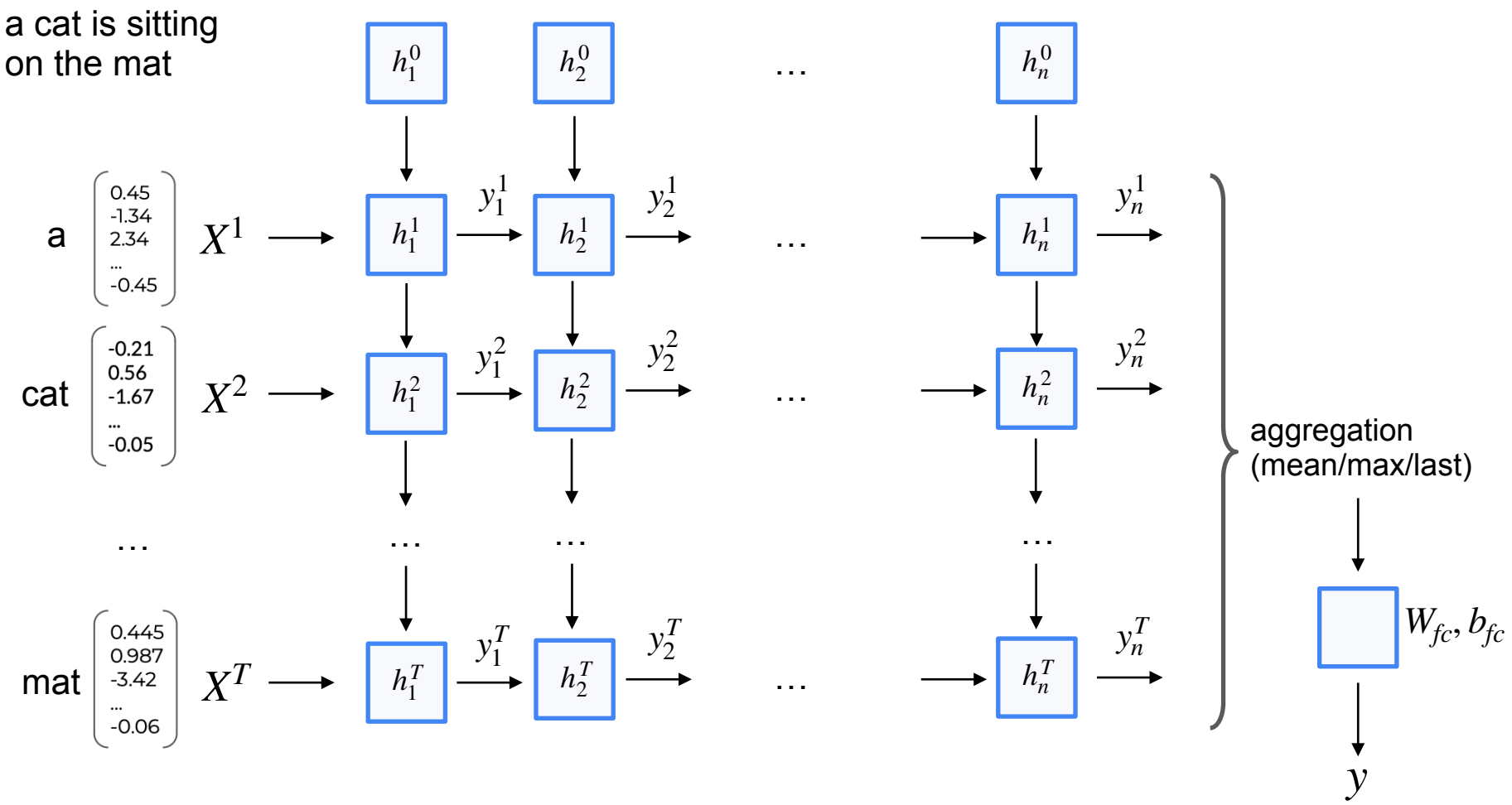
a cat is sitting
on the mat

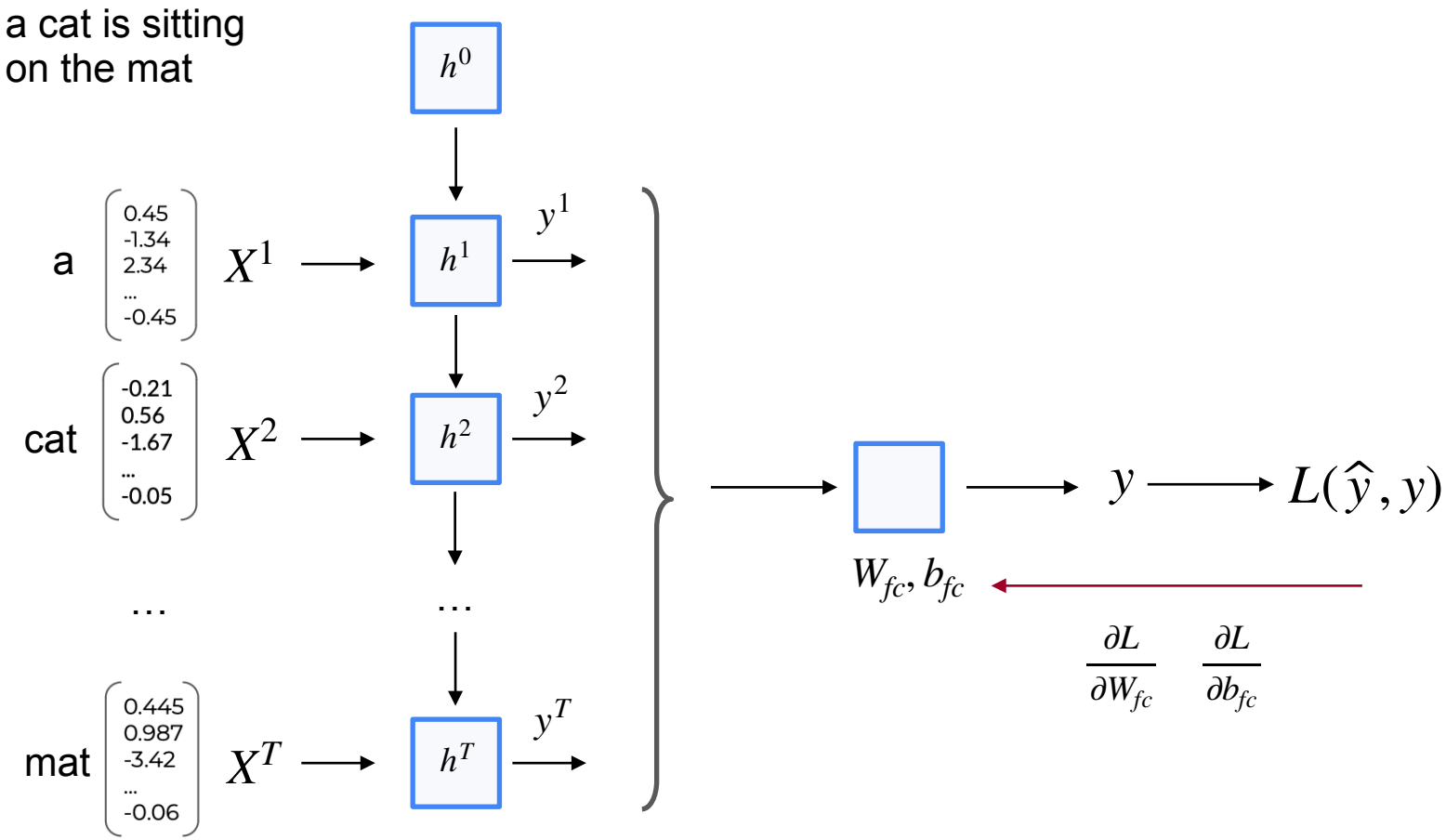


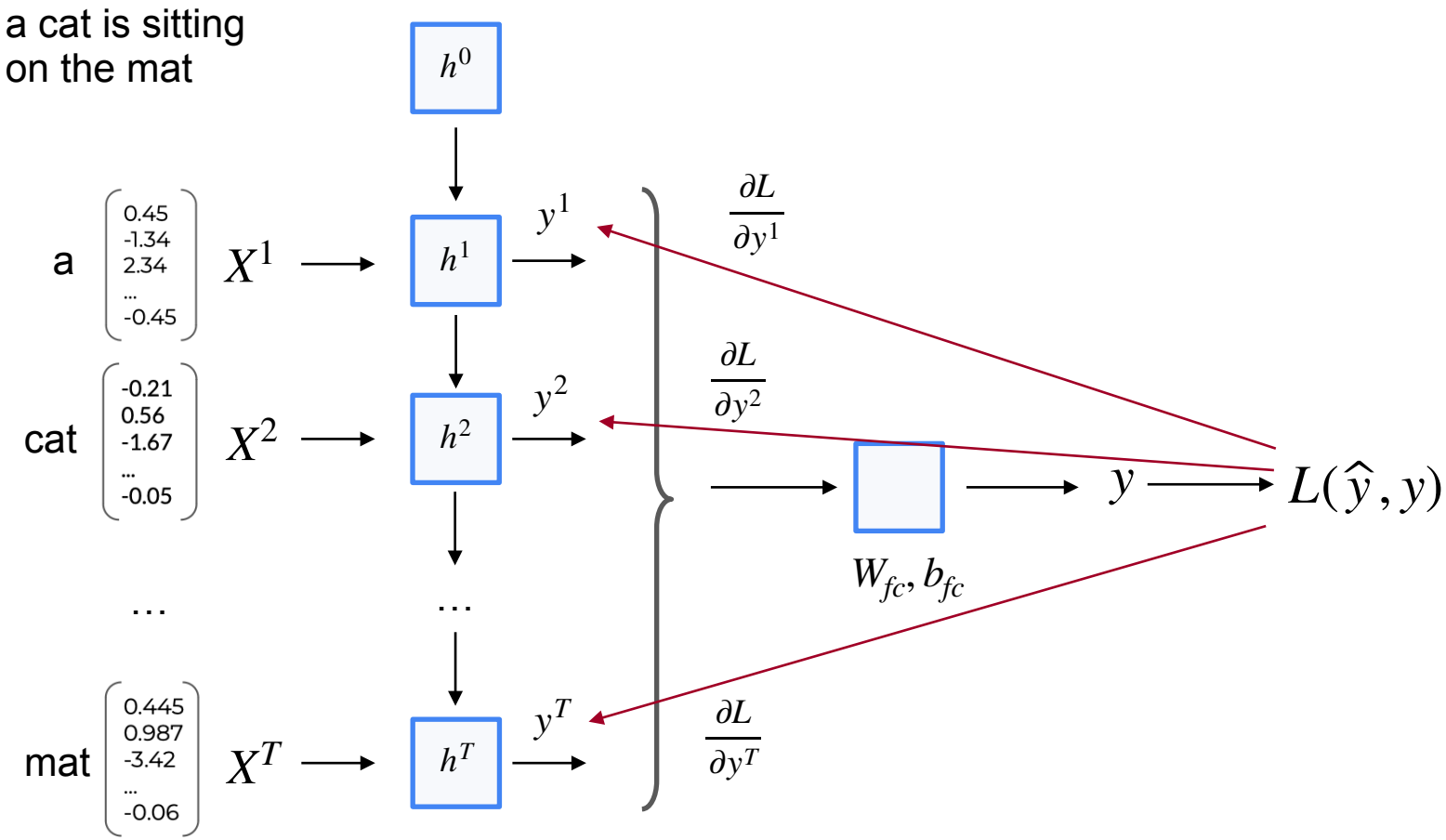
a cat is sitting
on the mat

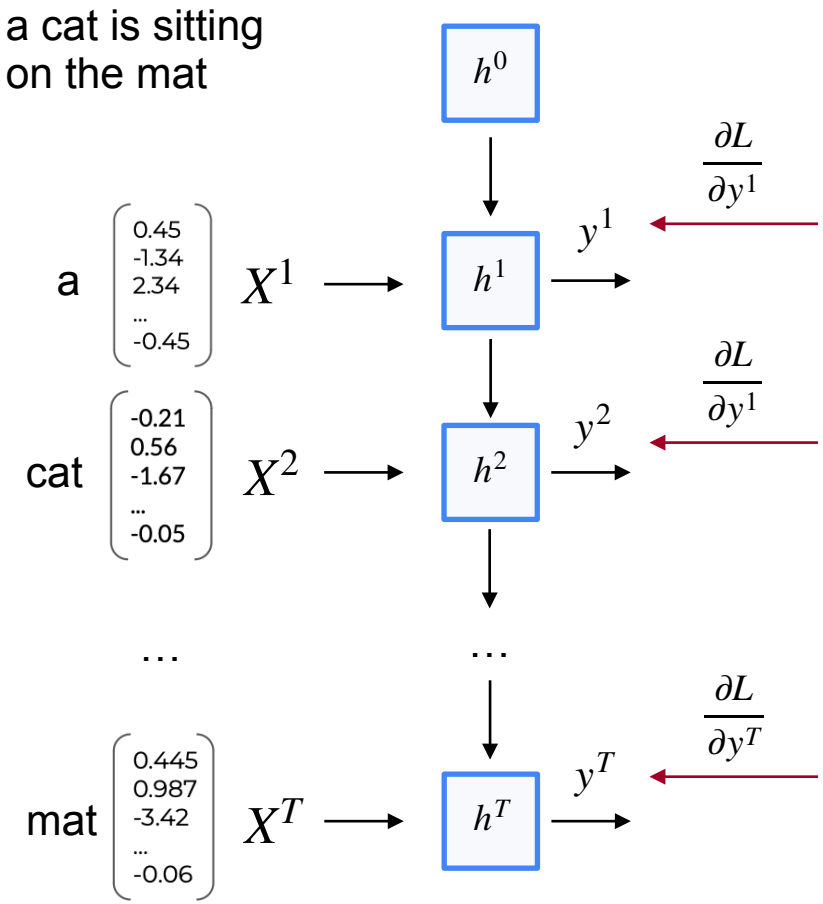



a cat is sitting
on the mat
a
0.45
-1.34
2.34
...
-0.45
cat
-0.21
0.56
-1.67
...
-0.05
mat
0.445
0.987
-3.42
...
-0.06
X^1
X^2
X^T
h_1^0
h_2^0
h_n^0
h_1^1
h_2^1
h_n^1
h_1^2
h_2^2
h_n^2
h_1^T
h_2^T
h_n^T
y_1^1
y_2^1
y_n^1
y_1^2
y_2^2
y_n^2
y_1^T
y_2^T
y_n^T


a cat is sitting on the mat
a
cat
mat
0.45
-1.34
2.34
...
-0.45
-0.21
0.56
-1.67
...
-0.05
0.445
0.987
-3.42
...
-0.06
X^1
X^2
X^T
h_1^0
h_2^0
h_n^0
h_1^1
h_2^1
h_n^1
h_1^2
h_2^2
h_n^2
h_1^T
h_2^T
h_n^T
y_1^1
y_2^1
y_n^1
y_1^2
y_2^2
y_n^2
y_1^T
y_2^T
y_n^T
aggregation
(mean/max/last)
W_{fc}, b_{fc}
y


a cat is sitting
on the mat
h^0
a
0.45
-1.34
2.34
...
-0.45
X^1
h^1
y^1
cat
-0.21
0.56
-1.67
...
-0.05
X^2
h^2
y^2
...
...
mat
0.445
0.987
-3.42
...
-0.06
X^T
h^T
y^T
W_{fc}, b_{fc}
y
$L(\hat{y}, y)$
$\frac{\partial L}{\partial W_{fc}}$
$\frac{\partial L}{\partial b_{fc}}$

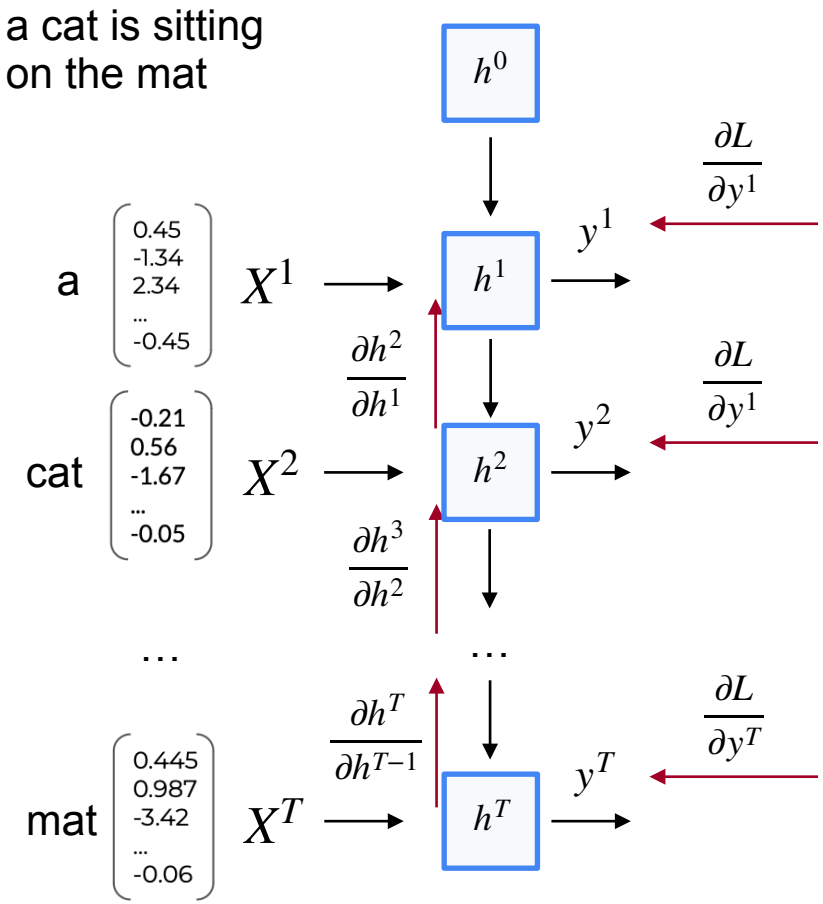

a cat is sitting
on the mat
h^0
a
0.45
-1.34
2.34
...
-0.45
X^1
h^1
y^1
$\frac{\partial L}{\partial y^1}$
cat
-0.21
0.56
-1.67
...
-0.05
X^2
h^2
y^2
$\frac{\partial L}{\partial y^2}$
W_{fc}, b_{fc}
y
$L(\hat{y}, y)$
...
...
mat
0.445
0.987
-3.42
...
-0.06
X^T
h^T
y^T
$\frac{\partial L}{\partial y^T}$



Hidden state on timestamp $t+1$ depends on hidden state on timestamp t

$$h^{t+1} = \sigma(WX^{t+1} + Uh^t + b_h)$$

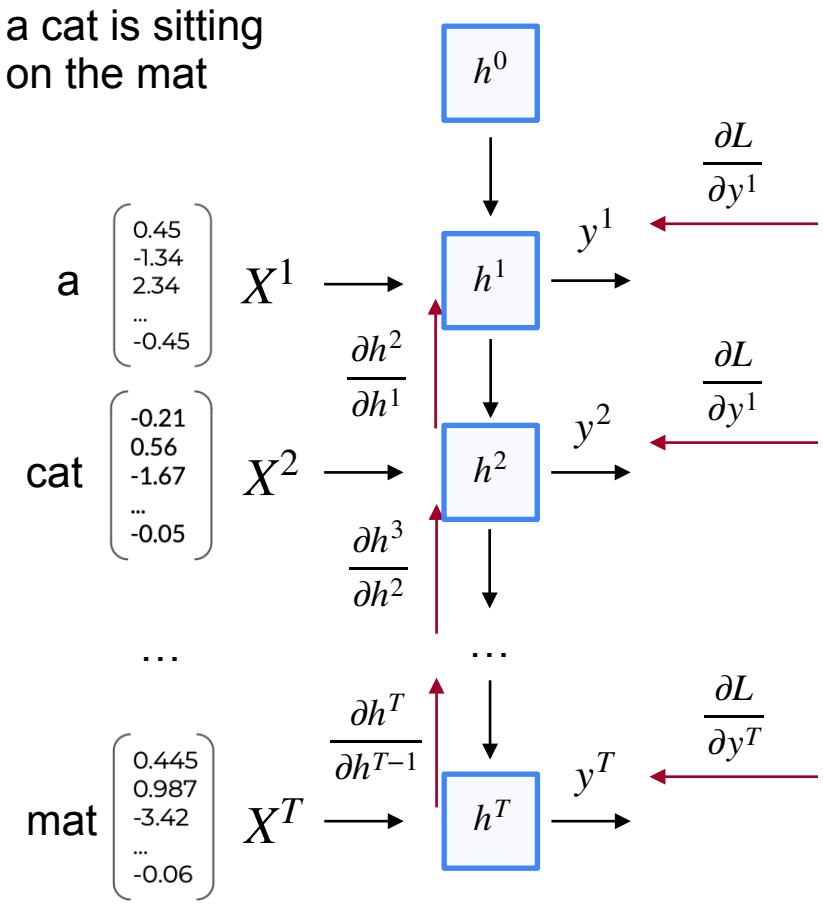
So $\frac{\partial L}{\partial h^t}$ is derived from all the $\frac{\partial L}{\partial h^{t+i}}$



Hidden state on timestamp $t+1$ depends on hidden state on timestamp t

$$h^{t+1} = \sigma(WX^{t+1} + Uh^t + b_h)$$

So $\frac{\partial L}{\partial h^t}$ is derived from all the $\frac{\partial L}{\partial h^{t+i}}$



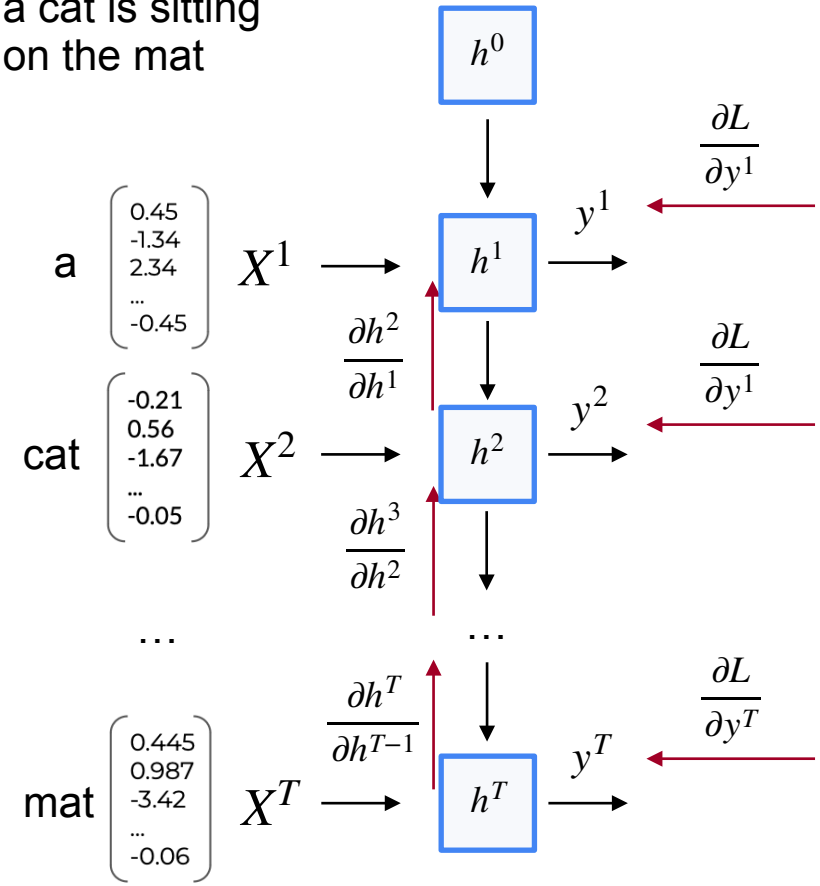
Hidden state on timestamp $t+1$ depends on hidden state on timestamp t

$$h^{t+1} = \sigma(WX^{t+1} + Uh^t + b_h)$$

So $\frac{\partial L}{\partial h^t}$ is derived from all the $\frac{\partial L}{\partial h^{t+i}}$

U, W, b_h depend on all the h^t, so to get $\frac{\partial L}{\partial U}, \frac{\partial L}{\partial W}, \frac{\partial L}{\partial b_h}$, we need to propagate gradients back through time

a cat is sitting
on the mat

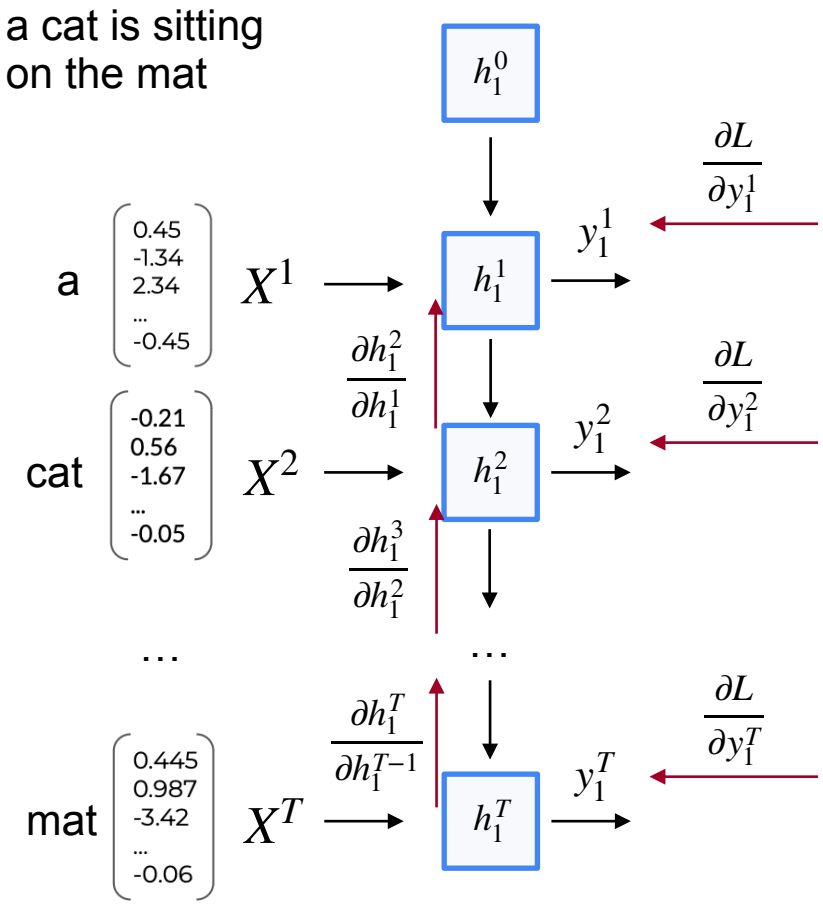


$$h^{t+1} = \sigma(WX^{t+1} + Uh^t + b_h)$$

$$\frac{\partial L}{\partial U} = \frac{\partial L}{\partial h^1}\frac{\partial h^1}{\partial U} + \frac{\partial L}{\partial h^2}\boxed{\frac{\partial h^2}{\partial U}} + \ldots + \frac{\partial L}{\partial h^T}\frac{\partial h^T}{\partial U}$$

$$\shortparallel$$

$$\frac{\partial h^2}{\partial h^1}\frac{\partial h^1}{\partial U}$$



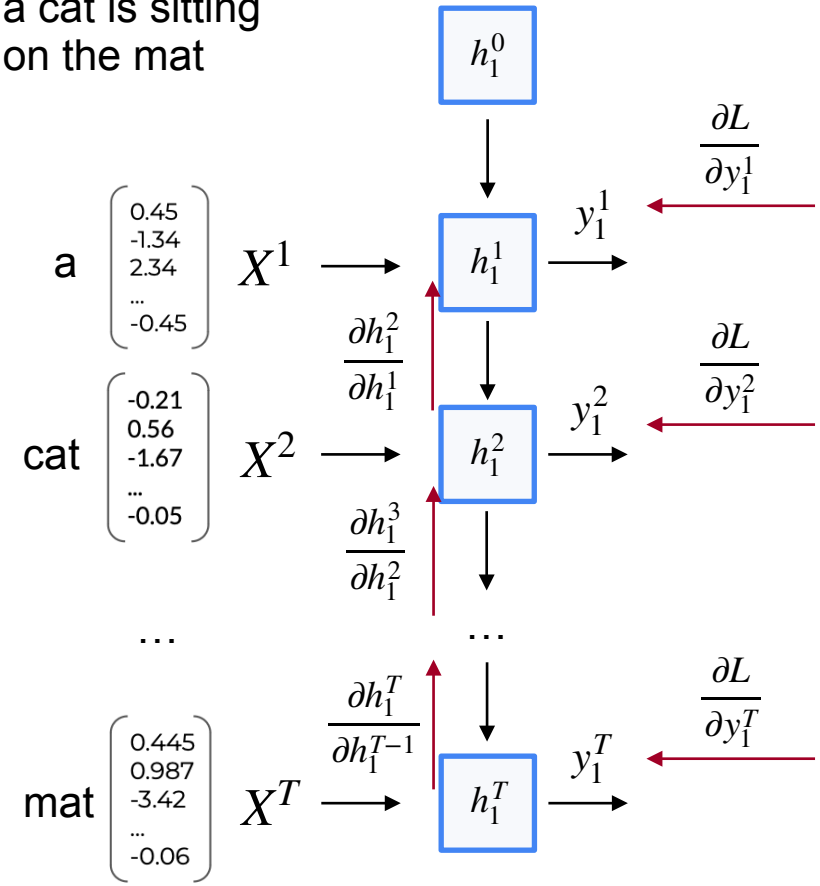
$$h^{t+1} = \sigma(WX^{t+1} + Uh^t + b_h)$$

$$\frac{\partial L}{\partial U} = \frac{\partial L}{\partial h^1}\frac{\partial h^1}{\partial U} + \frac{\partial L}{\partial h^2}\frac{\partial h^2}{\partial U} + \ldots + \frac{\partial L}{\partial h^T}\boxed{\frac{\partial h^T}{\partial U}}$$

$$\shortparallel$$

$$\frac{\partial h^T}{\partial h^{T-1}}\frac{\partial h^{T-1}}{\partial h^{T-2}}\cdots\frac{\partial h^2}{\partial h^1}\frac{\partial h^1}{\partial U}$$

a cat is sitting
on the mat



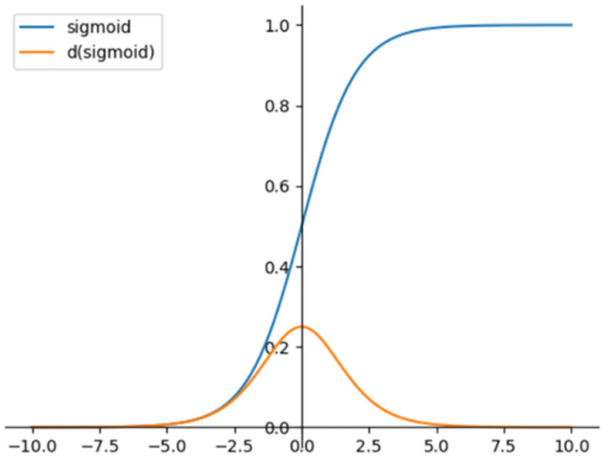
$$h^{t+1} = \sigma(WX^{t+1} + Uh^t + b_h)$$

$$\frac{\partial L}{\partial U} = \frac{\partial L}{\partial h^1}\frac{\partial h^1}{\partial U} + \frac{\partial L}{\partial h^2}\frac{\partial h^2}{\partial U} + \ldots + \frac{\partial L}{\partial h^T}\boxed{\frac{\partial h^T}{\partial U}}$$

$$\shortparallel$$

$$\frac{\partial h^T}{\partial h^{T-1}}\frac{\partial h^{T-1}}{\partial h^{T-2}}\cdots\frac{\partial h^2}{\partial h^1}\frac{\partial h^1}{\partial U}$$

There's derivative of σ
in every multiplier



$$h^{t+1} = \sigma(WX^{t+1} + Uh^t + b_h)$$

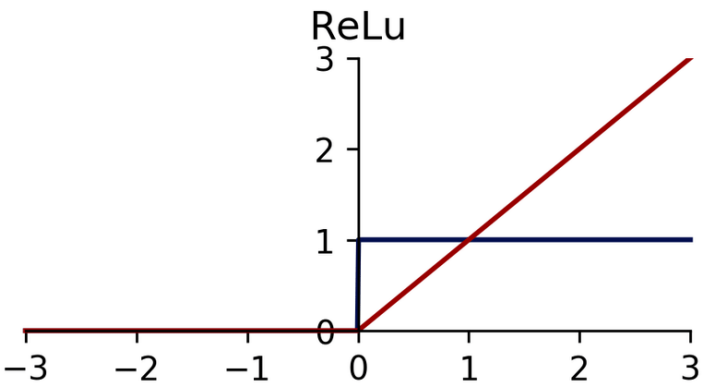
$$\frac{\partial L}{\partial U} = \frac{\partial L}{\partial h^1}\frac{\partial h^1}{\partial U} + \frac{\partial L}{\partial h^2}\frac{\partial h^2}{\partial U} + \ldots + \frac{\partial L}{\partial h^T}\boxed{\frac{\partial h^T}{\partial U}}$$

$$\shortparallel$$

$$\frac{\partial h^T}{\partial h^{T-1}}\frac{\partial h^{T-1}}{\partial h^{T-2}}\cdots\frac{\partial h^2}{\partial h^1}\frac{\partial h^1}{\partial U}$$

If we use *Sigmoid* as σ, we risk to get a *vanishing gradient* problem

There's derivative of σ in every multiplier



$$h^{t+1} = \sigma(WX^{t+1} + Uh^t + b_h)$$

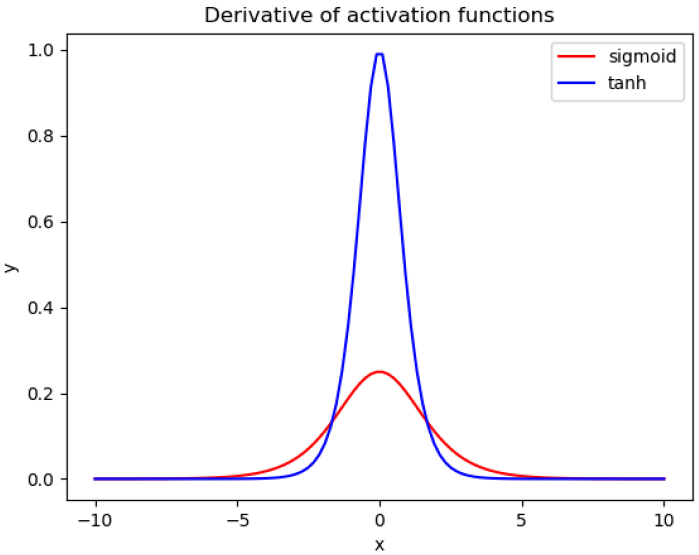
$$\frac{\partial L}{\partial U} = \frac{\partial L}{\partial h^1}\frac{\partial h^1}{\partial U} + \frac{\partial L}{\partial h^2}\frac{\partial h^2}{\partial U} + \ldots + \frac{\partial L}{\partial h^T}\boxed{\frac{\partial h^T}{\partial U}}$$

$$\shortparallel$$

$$\frac{\partial h^T}{\partial h^{T-1}}\frac{\partial h^{T-1}}{\partial h^{T-2}}\cdots\frac{\partial h^2}{\partial h^1}\frac{\partial h^1}{\partial U}$$

If we use *ReLU* as σ,
we risk to get an *exploding gradient* problem

There's derivative of σ
in every multiplier



Compromise is using *Tanh* as σ

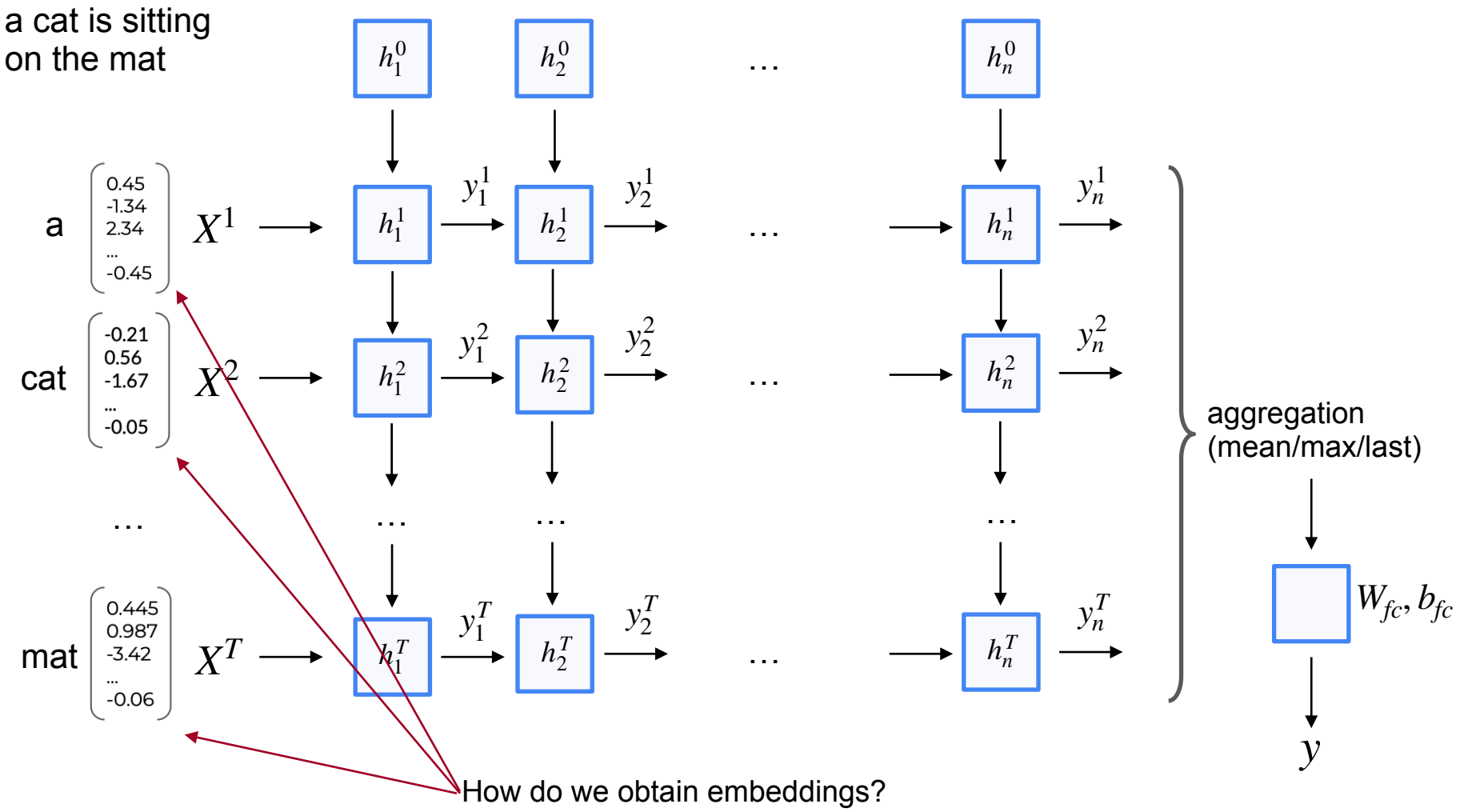
$$h^{t+1} = \sigma(WX^{t+1} + Uh^t + b_h)$$

$$\frac{\partial L}{\partial U} = \frac{\partial L}{\partial h^1}\frac{\partial h^1}{\partial U} + \frac{\partial L}{\partial h^2}\frac{\partial h^2}{\partial U} + \ldots + \frac{\partial L}{\partial h^T}\boxed{\frac{\partial h^T}{\partial U}}$$

$$\shortparallel$$

$$\frac{\partial h^T}{\partial h^{T-1}}\frac{\partial h^{T-1}}{\partial h^{T-2}}\cdots\frac{\partial h^2}{\partial h^1}\frac{\partial h^1}{\partial U}$$

There's derivative of σ in every multiplier


a cat is sitting on the mat
a
cat
mat
0.45
-1.34
2.34
...
-0.45
-0.21
0.56
-1.67
...
-0.05
0.445
0.987
-3.42
...
-0.06
X^1
X^2
X^T
h_1^0
h_2^0
h_n^0
h_1^1
h_2^1
h_n^1
h_1^2
h_2^2
h_n^2
h_1^T
h_2^T
h_n^T
y_1^1
y_2^1
y_n^1
y_1^2
y_2^2
y_n^2
y_1^T
y_2^T
y_n^T
aggregation (mean/max/last)
W_{fc}, b_{fc}
y
How do we obtain embeddings?

Building vocabulary

1. Build a vocabulary of tokens of a size n. For example, select n most frequently used words from the training data.

a	0
an	1
apple	2
…	…
cat	356
…	…
tree	18546
…	…

embedding of the token “cat”

2. Assign each token a trainable embedding of a size d. We get **embedding matrix**

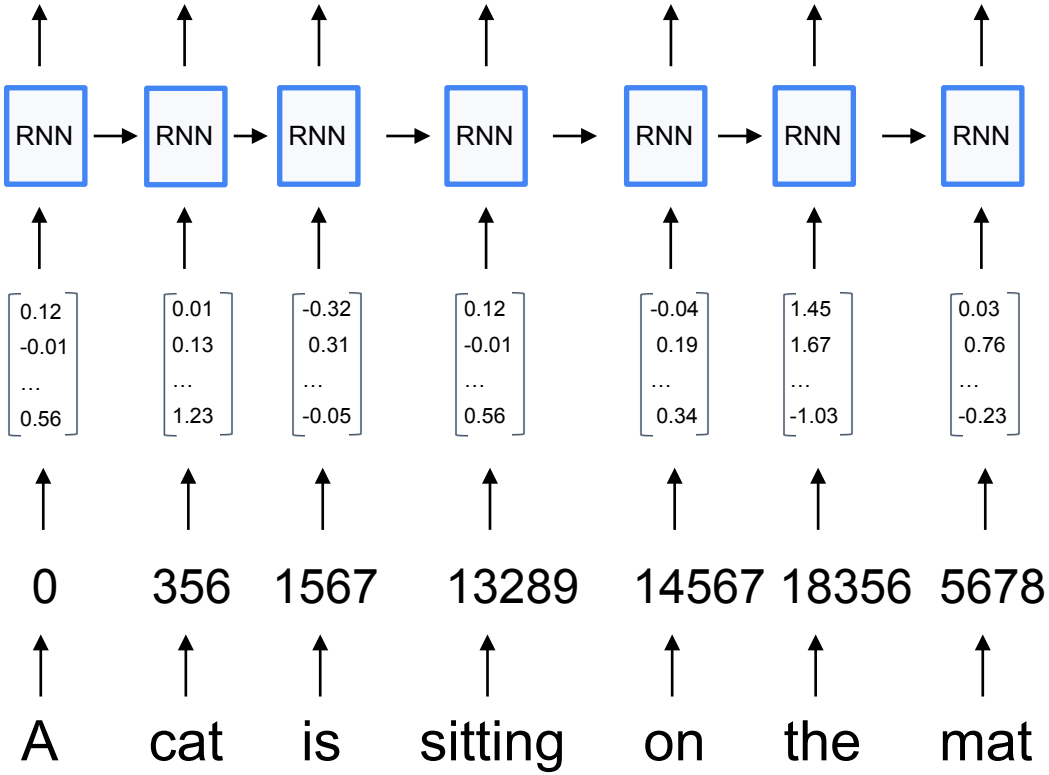
All the elements of the embedding matrix will be trainable

a	an	apple	…	cat	…	tree	…
0.01	0.23	-0.56	…	0.45	…	-1.34	…
0.54	0.11	0.24	…	0.40	…	-0.43	…
-0.12	-0.03	0.12	…	-0.31	…	0.04	…
			…				
-0.05	0.23	0.22	…	-1.34	…	1.44	…

d

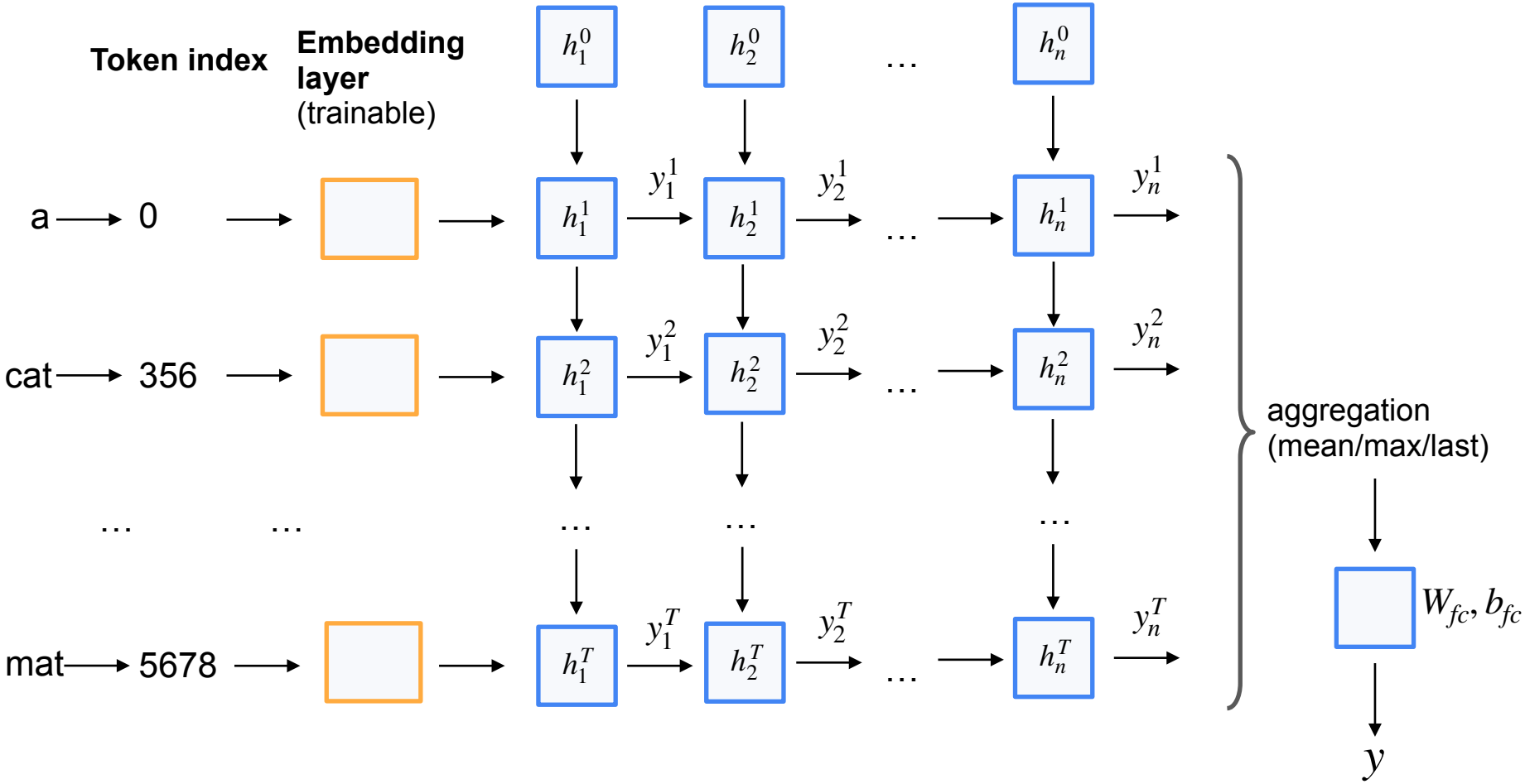
n

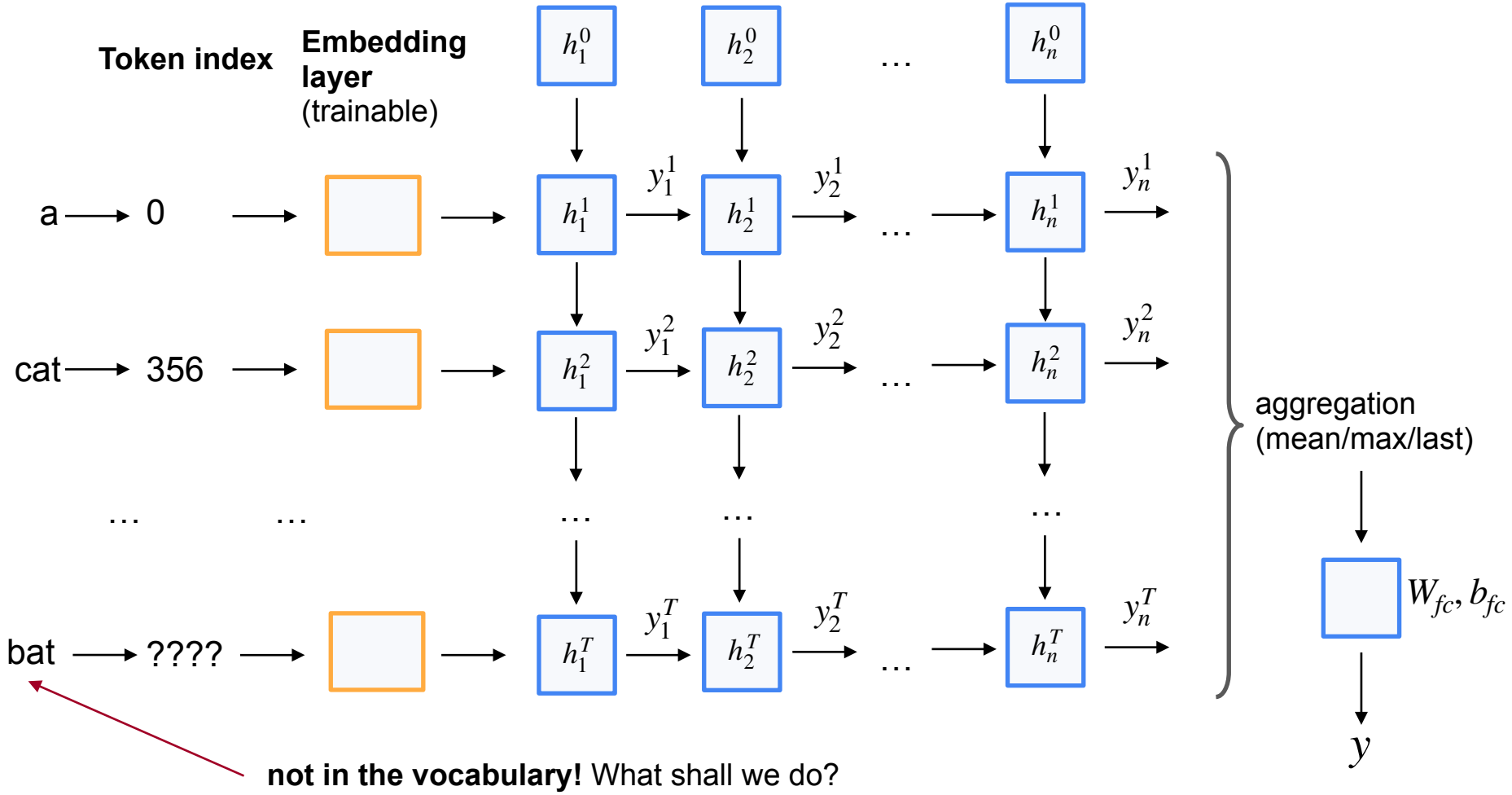
What happens during training/inference:



Get embedding by index from the embedding matrix

Get index of token in the vocabulary


Token index
Embedding layer
(trainable)
a
0
cat
356
mat
5678
h_1^0
h_2^0
h_n^0
h_1^1
h_2^1
h_n^1
h_1^2
h_2^2
h_n^2
h_1^T
h_2^T
h_n^T
y_1^1
y_2^1
y_n^1
y_1^2
y_2^2
y_n^2
y_1^T
y_2^T
y_n^T
aggregation
(mean/max/last)
W_{fc}, b_{fc}
y


Token index
Embedding layer
(trainable)
a
0
cat
356
bat
????
h_1^0
h_2^0
h_n^0
h_1^1
h_2^1
h_n^1
h_1^2
h_2^2
h_n^2
h_1^T
h_2^T
h_n^T
y_1^1
y_2^1
y_n^1
y_1^2
y_2^2
y_n^2
y_1^T
y_2^T
y_n^T
aggregation
(mean/max/last)
W_{fc}, b_{fc}
y
not in the vocabulary! What shall we do?

Building vocabulary

1. Build a vocabulary of tokens of a size n. For example, select n most frequently used words from the training data.
Add new token representing unknown words <\unk>

2. Assign each token a trainable embedding of a size d. We get **embedding matrix**

All the elements of the embedding matrix will be trainable
<\unk> is usually also trainable

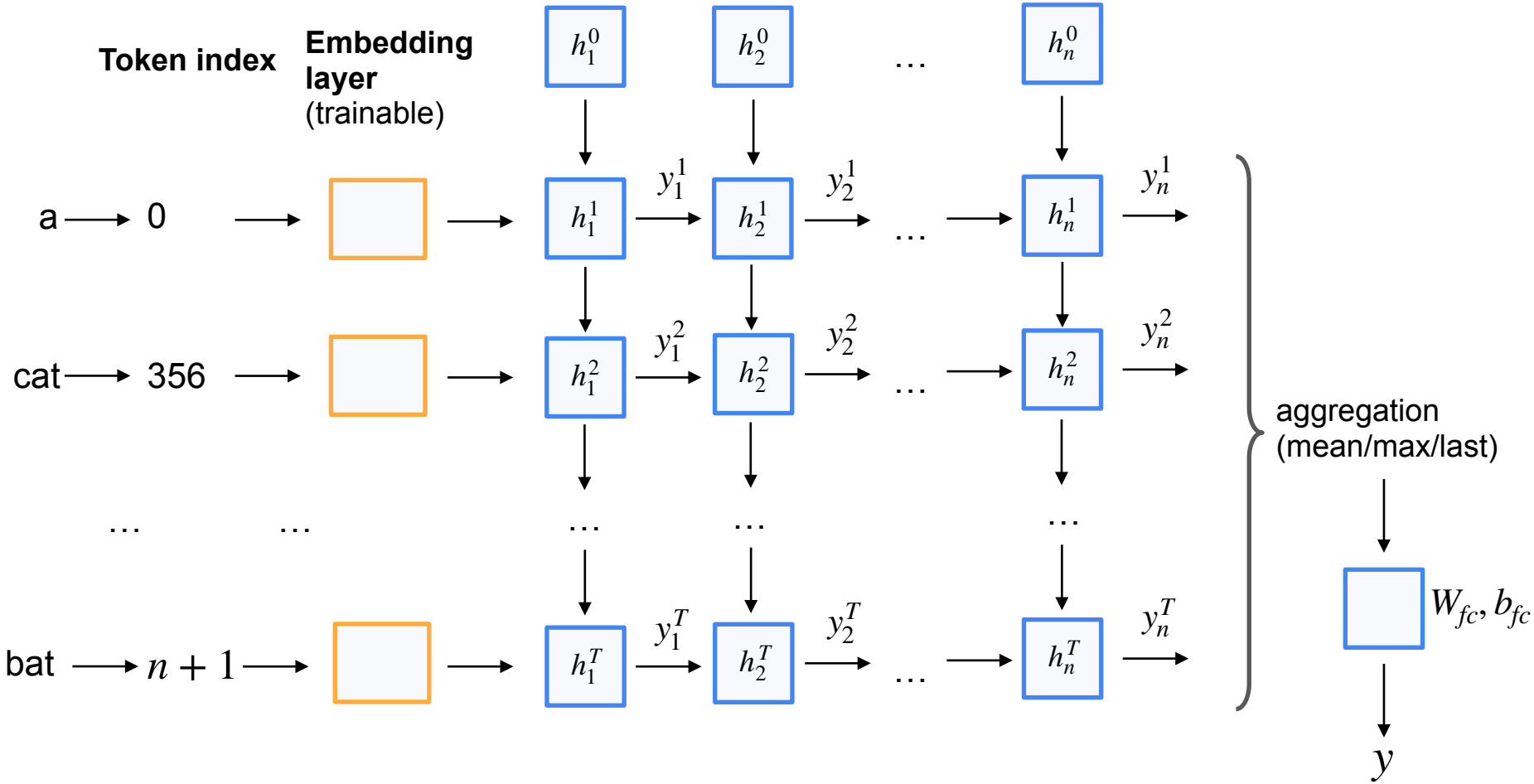
a	0
an	1
apple	2
…	…
cat	356
…	…
tree	18546
…	…
<\unk>	$n+1$

embedding of the token “cat”

a	an	apple	…	cat	…	tree	…	<\unk>
0.01	0.23	-0.56	…	0.45	…	-1.34	…	0.12
0.54	0.11	0.24	…	0.40	…	-0.43	…	0.34
-0.12	-0.03	0.12	…	-0.31	…	0.04	…	-0.56
			…					
-0.05	0.23	0.22	…	-1.34	…	1.44	…	-1.23

d

$n+1$


Token index
Embedding layer
(trainable)
a
0
cat
356
bat
$n+1$
h_1^0
h_2^0
h_n^0
h_1^1
h_2^1
h_n^1
h_1^2
h_2^2
h_n^2
h_1^T
h_2^T
h_n^T
y_1^1
y_2^1
y_n^1
y_1^2
y_2^2
y_n^2
y_1^T
y_2^T
y_n^T
aggregation
(mean/max/last)
W_{fc}, b_{fc}
y

Tokenizing texts

Previously we've been building the vocabulary using **words** as tokens.

It has some disadvantages:

Tokenizing texts

Previously we've been building the vocabulary using **words** as tokens.

It has some disadvantages:

- Unknown words (partially solved by adding <\unk> token)
- There's tons of word variations in the language, so vocabulary size should be really big to cover most of the needed words
- Different forms of one word will get different embeddings
- Misspelled words, names, surnames go to <\unk>

color colour

great greater greatest

create creates created creating

Char-level tokenization

Advantanges:

- Suffers less from unknown words

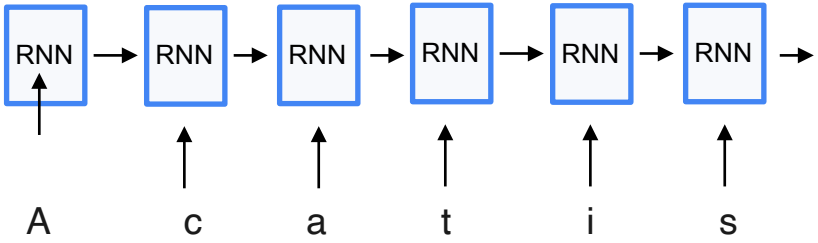
(ô will still suffer if we've built only English vocabulary)

Disadvantages:

- Sequence lengths become enormous.

It slows down training and inference,

increases compute cost.



In Transformers (we'll learn then in the next class!)

it also decreases effective context length

BPE (Byte-Pair Encoding)

Overall idea:

- Iteratively build vocabulary out of most frequently appearing sets of characters
- Use bytes (e.g. ASCII codes) instead of raw characters

BPE (Byte-Pair Encoding)

Overall idea:

- Iteratively build vocabulary out of most frequently appearing sets of characters
- Use bytes (e.g. ASCII codes) instead of raw characters

First, let's consider this idea based on raw English chars

BPE (Byte-Pair Encoding)

How many times the word is present in the training set

Suppose we have the following training set:

```
("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)
```

We begin with the basic char-level vocabulary:

```
["b", "g", "h", "n", "p", "s", "u"]
```

https://huggingface.co/learn/llm-course/chapter6/5

BPE (Byte-Pair Encoding)

How many times the word is present in the training set

Suppose we have the following training set:

("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)

We begin with the basic char-level vocabulary:

["b", "g", "h", "n", "p", "s", "u"]

We go through our texts in the training set and consider all possible consecutive pairs of existing vocab tokens and their frequencies:

"hu"	"ug"	"pu"	"un"	"bu"	"gs"
15	20	17	16	4	5

https://huggingface.co/learn/llm-course/chapter6/5

BPE (Byte-Pair Encoding)

How many times the word is present in the training set

Suppose we have the following training set:

("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)

We add the most frequent pair into our vocabulary and formulate the first merging rule:

["b", "g", "h", "n", "p", "s", “u”, “ug”]

“u” → “ug”

"g" → “ug”

We go through our texts in the training set again and consider all possible consecutive pairs of existing vocab tokens and their frequencies:

["hu",	~~“ug”~~,	"pu",	"un",	"bu",	"gs",	"hug",	"pug",	"ugs"]
0		12	16	4	0	15	5	5

https://huggingface.co/learn/llm-course/chapter6/5

BPE (Byte-Pair Encoding)

How many times the word is present in the training set

Suppose we have the following training set:

("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)

We add the most frequent pair into our vocabulary and formulate the first merging rule:

["b", "g", "h", "n", "p", "s", “u”, “ug”, “un”]

“u” → “ug”
"g" → “ug”

“u” → “un”
“n" → “un”

We go through our texts in the training set again and consider all possible consecutive pairs of existing vocab tokens and their frequencies:

["hu",	~~“ug”~~,	"pu",	~~"un"~~,	"bu",	"gs",	"hug",	"pug",	"ugs",	"pun",	"bun"]
0		0		0	0	15	5	5	12	4

https://huggingface.co/learn/llm-course/chapter6/5

BPE (Byte-Pair Encoding)

How many times the word is present in the training set

Suppose we have the following training set:

("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)

We add the most frequent pair into our vocabulary and formulate the first merging rule:

["b", "g", "h", "n", "p", "s", “u”, “ug”, “un”, “hug”]

“u” → “ug”
"g" → “ug”

“u” → “un”
“n” → “un”

"h" → "hug"
“ug” → "hug"

We go through our texts in the training set again and consider all possible consecutive pairs of existing vocab tokens and their frequencies:

["hu",	~~“ug”~~,	"pu",	~~"un"~~,	"bu",	"gs",	"hug",	"pug",	"ugs",	"pun",	"bun"]
0		0		0	0	15	5	5	12	4

https://huggingface.co/learn/llm-course/chapter6/5

BPE (Byte-Pair Encoding)

During inference we split the text into characters and apply learned rules to construct tokens

"hugs" → "h" "u" "g" "s" → "h" "ug" "s" → "hug" "s"

“u” → “ug”
"g" → “ug”

“u” → “un”
“n" → “un”

"h" → "hug"
“ug” → "hug"

https://huggingface.co/learn/llm-course/chapter6/5

BPE (Byte-Pair Encoding)

Byte-pair: let's use bytes (e.g. UTF-8) instead of chars!

Every language can be encoded using UTF-8 bytes, so we won't need any <\unk> tokens.

BPE will start with the vocabulary of raw bytes.

"A" → [65]

"hello" → [104, 101, 108, 108, 111]

" " → [32]

"\n" → [10]

"é" → [195, 169]

000		(nul)	016	►	(dle)	032	sp	048	0̇	064	@	080	P		
001	☺	(soh)	017	◄	(dc1)	033	!	049	1	065	A	081	Q		
002	☻	(stx)	018	↕	(dc2)	034	"	050	2	066	B	082	R		
003	♥	(etx)	019	‼	(dc3)	035	#	051	3	067	C	083	S		
004	♦	(eot)	020	¶	(dc4)	036	$	052	4	068	D	084	T		
005	♣	(enq)	021	§	(nak)	037	%	053	5	069	E	085	U		
006	♠	(ack)	022	▬	(syn)	038	&	054	6	070	F	086	V		
007	•	(bel)	023	↨	(etb)	039	'	055	7	071	G	087	W		
008	◘	(bs)	024	↑	(can)	040	(	056	8	072	H	088	X		
009		(tab)	025	↓	(em)	041	)	057	9	073	I	089	Y		
010		(lf)	026		(eof)	042	*	058	:	074	J	090	Z		
011	♂	(vt)	027	←	(esc)	043	+	059	;	075	K	091	[		
012	♀	(np)	028	∟	(fs)	044	,	060	<	076	L	092	\		
013		(cr)	029	↔	(gs)	045	-	061	=	077	M	093	]		

https://huggingface.co/learn/llm-course/chapter6/5

BPE (Byte-Pair Encoding)

Advantages:

- No need for <\unk>
- Usage of vocabulary is effective (based on training data)
- BPE provides middle-ground between char-level and word-level tokenization

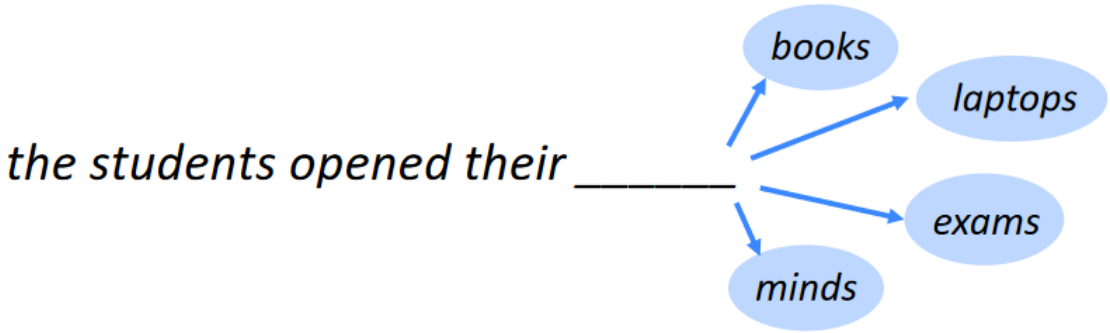
Notes:

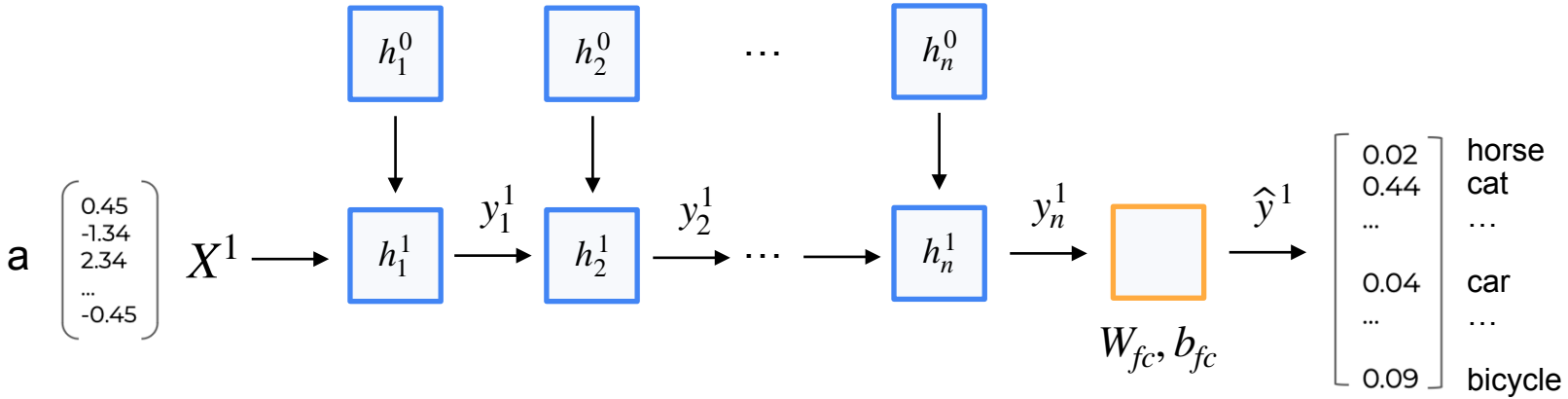
- BPE tokens are not necessarily interpretable
- BPE tokenization depends on the training data
- Some symbols may be represented by more bytes that others (e.g. cyrillic)

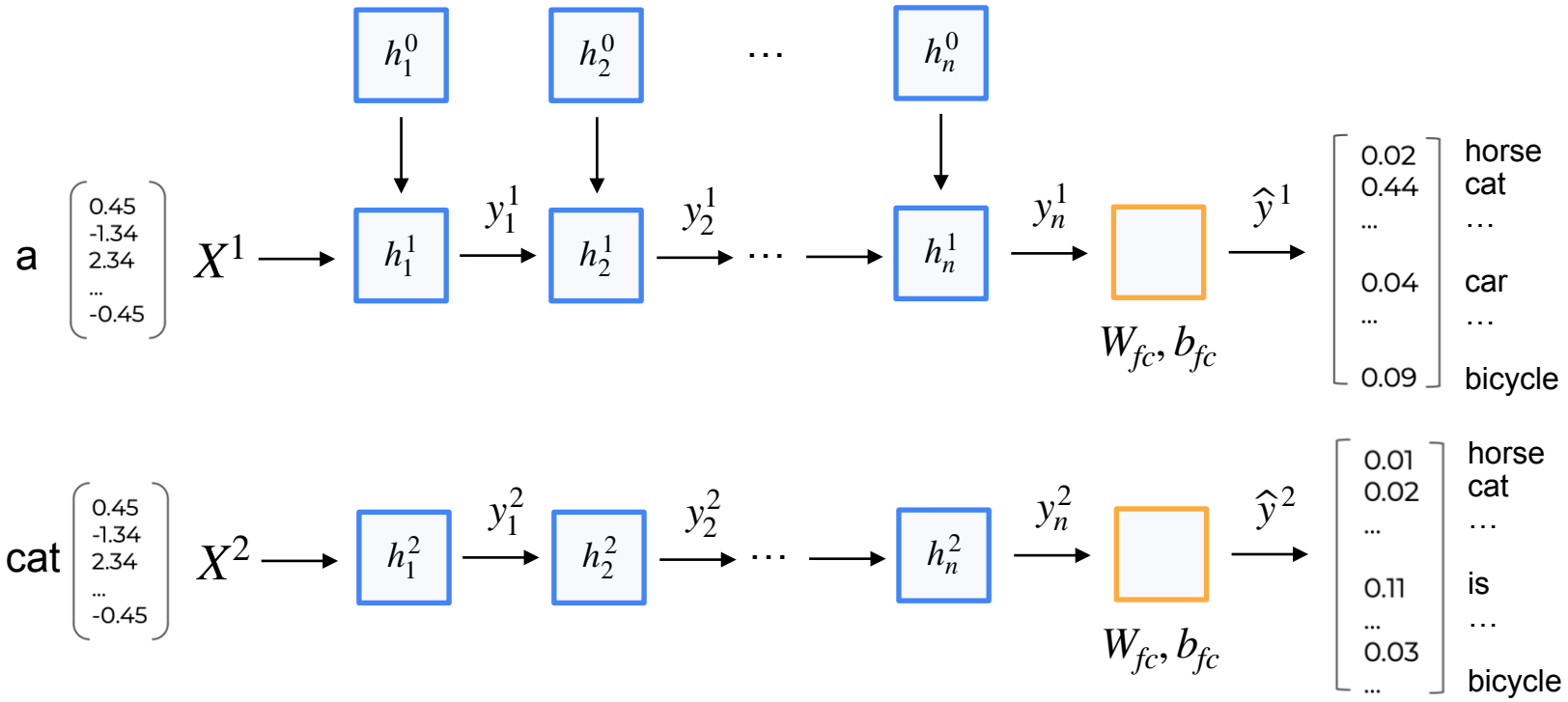
https://huggingface.co/learn/llm-course/chapter6/5

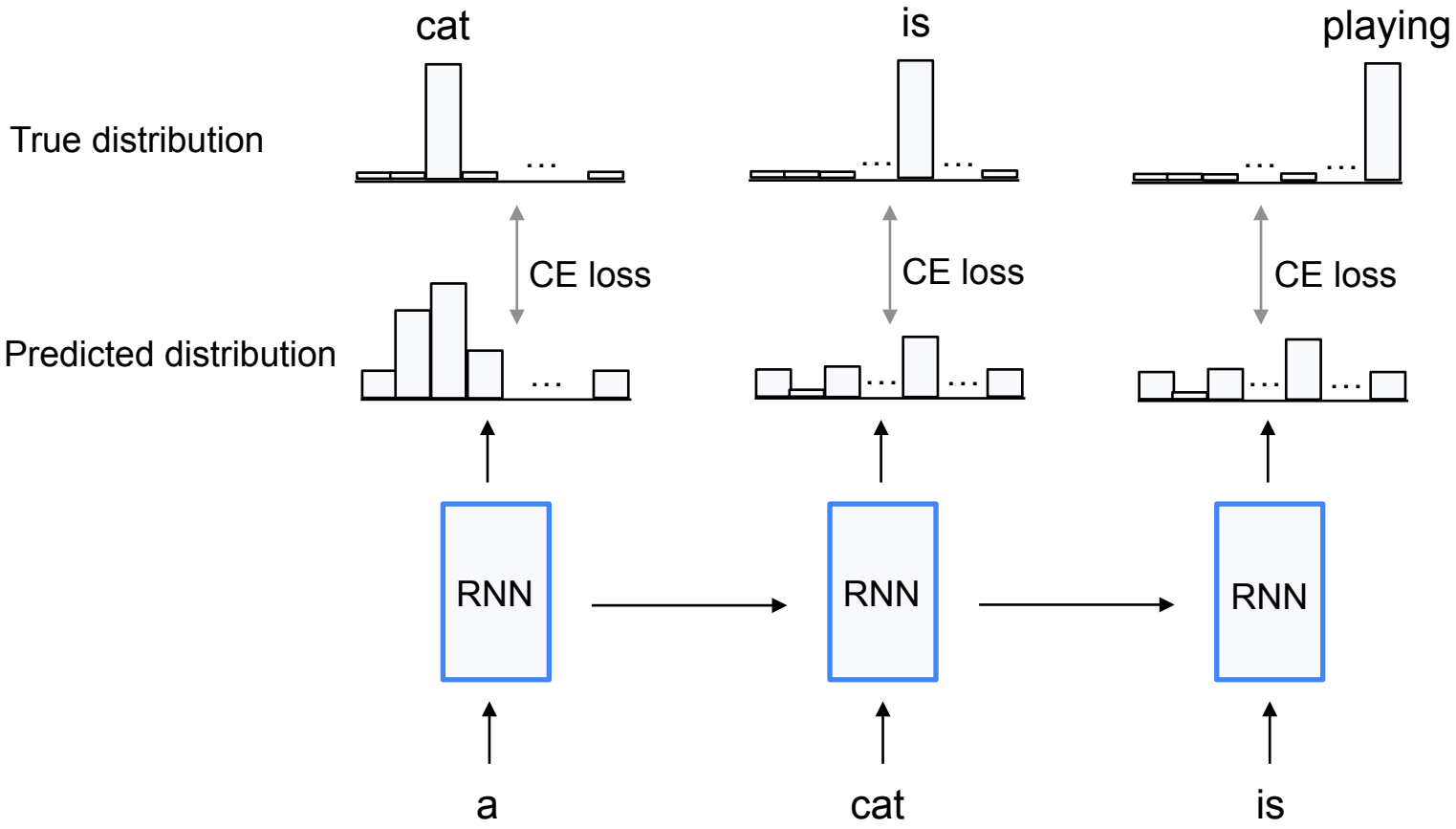
Language Modelling

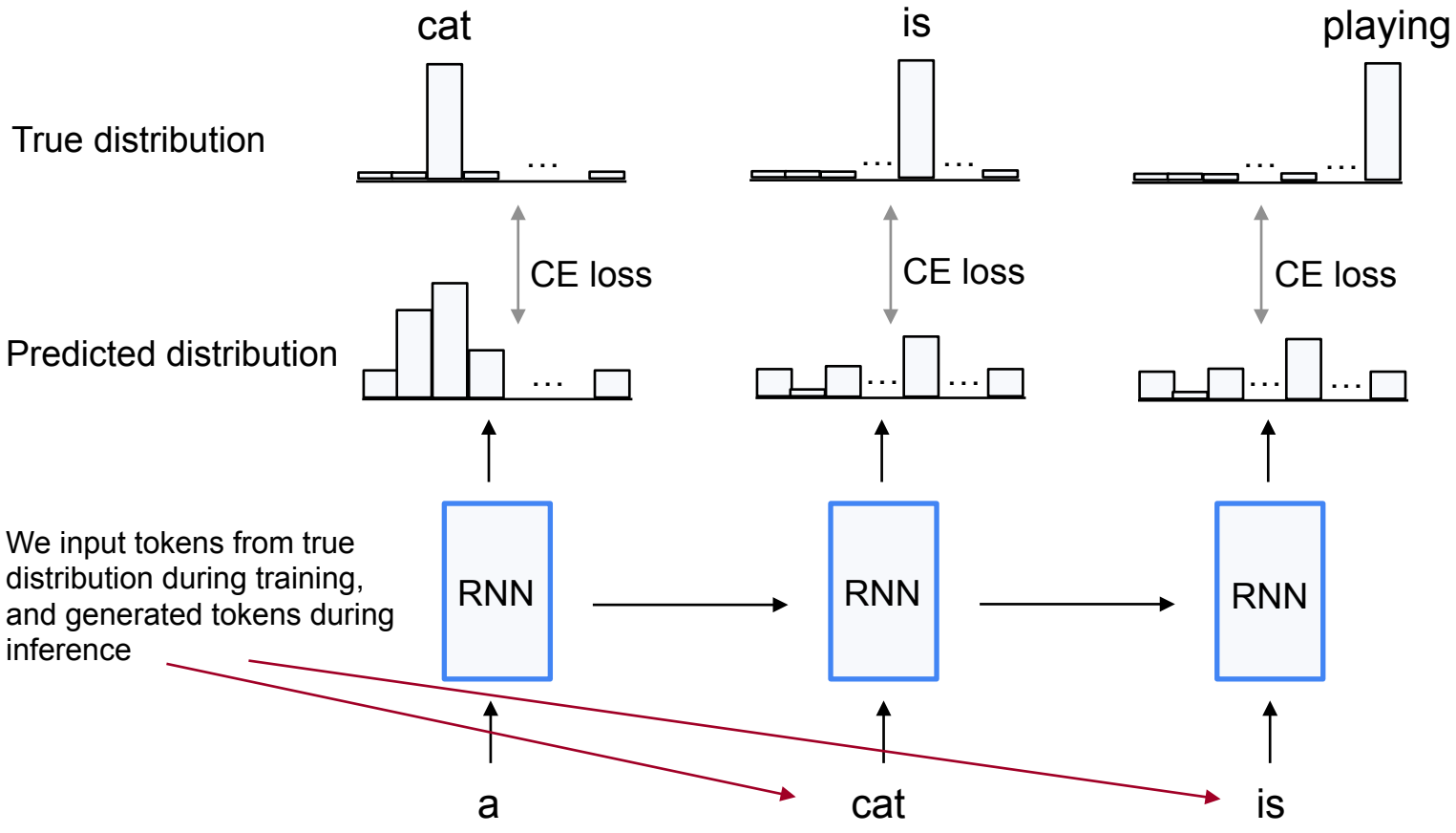
Language Modelling is the task of modelling the probability of the next token




a
0.45
-1.34
2.34
...
-0.45
X^1
h_1^0
h_2^0
h_n^0
h_1^1
h_2^1
h_n^1
y_1^1
y_2^1
y_n^1
W_{fc}, b_{fc}
$\hat{y}^1$
0.02
0.44
...
0.04
...
0.09
horse
cat
car
bicycle


h_1^0
h_2^0
$\cdots$
h_n^0
a
0.45
-1.34
2.34
...
-0.45
X^1
h_1^1
y_1^1
h_2^1
y_2^1
$\cdots$
h_n^1
y_n^1
$\hat{y}^1$
W_{fc}, b_{fc}
0.02
0.44
...
0.04
...
0.09
horse
cat
...
car
...
bicycle
cat
0.45
-1.34
2.34
...
-0.45
X^2
h_1^2
y_1^2
h_2^2
y_2^2
$\cdots$
h_n^2
y_n^2
$\hat{y}^2$
W_{fc}, b_{fc}
0.01
0.02
...
0.11
...
0.03
...
horse
cat
...
is
...
bicycle


cat
is
playing
True distribution
CE loss
CE loss
CE loss
Predicted distribution
RNN
RNN
RNN
a
cat
is

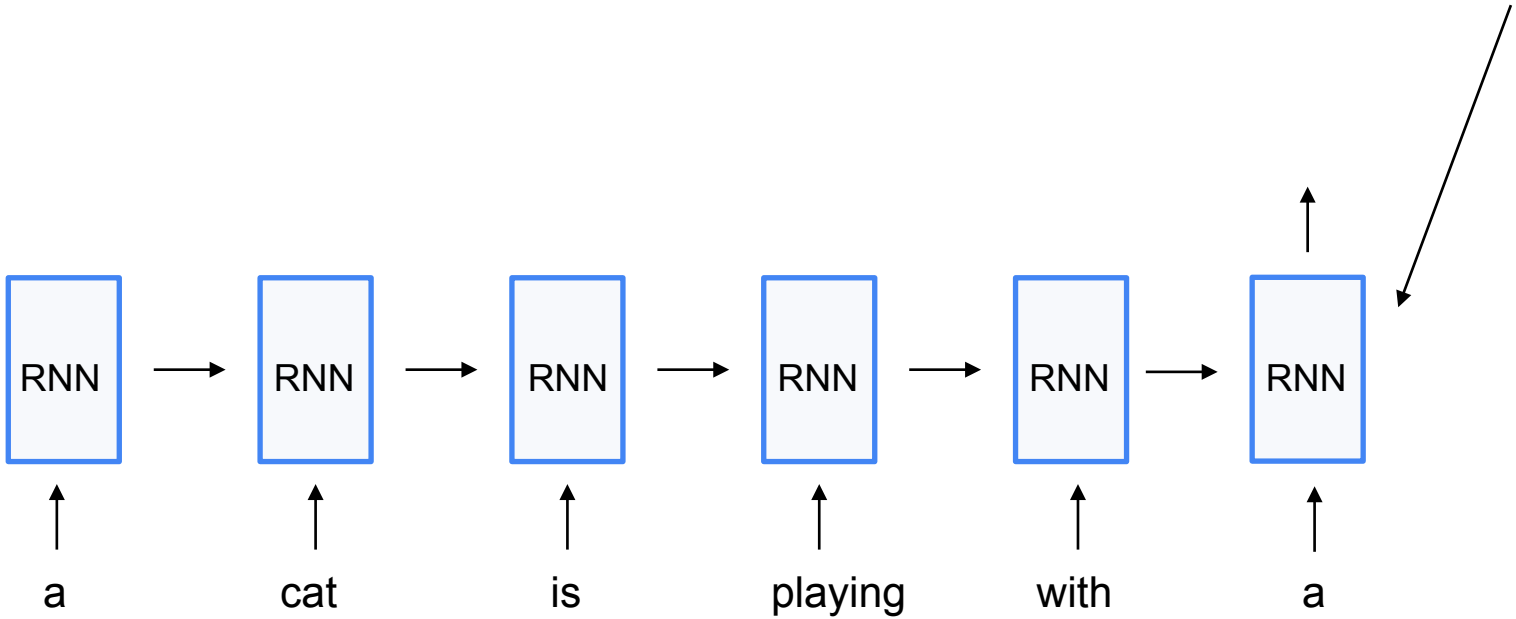

cat
is
playing
True distribution
CE loss
CE loss
CE loss
Predicted distribution
We input tokens from true distribution during training, and generated tokens during inference
RNN
RNN
RNN
a
cat
is

RNN hidden state update:

$$h^t = \sigma(WX^t + Uh^{t-1} + b_h)$$

This hidden state of RNN should contain all the information about sentence

RNNs are prone to forgetting



RNN forgetting

How to solve:

- Better recurrent units, e.g. GRU/LSTM
- Bidirectional RNN (for encoders only)
- Attention mechanism! (In the next class) ->

go to Transformers instead of RNNs =)

Recap

In this lecture we've learned:

- Recurrent Neural Networks (RNNs)
    - How recurrent layer works
    - How RNNs are trained
    - How tokenization is performed
    - How to build language modelling with RNN